Linux System Administration (E1ITA307)

Linux Infrastructure Deployment Project Report

On

LINUX STORAGE MONITORING SYSTEM

Submitted by

LAVANYA CH (2411021061493)

KISHORE V (2411021060180)

SHAIK BABAVALI (2411021060997)

G K DHARSHAN RAJ (2411021060771)

In partial fulfillment of the award of the degree of

BACHELOR OF TECHNOLOGY

In

Information Technology

Under the Supervision of

Prof. Ananthanagu U

ALLIANCE SCHOOL OF ADVANCED COMPUTING

ALLIANCE UNIVERSITY

BENGALURU

Oct 2026

DEPARTMENT OF INFORMATION TECHNOLOGY

ALLIANCE SCHOOL OF ADVANCED COMPUTING

CERTIFICATE

This is to certify that the Project work entitled **"Linux Storage Monitoring System"** submitted by Lavanya CH [2411021061493], Kishore V [2411021060180], Shaik BabaVali [2411021060997] and G K Darshan Raj [2411021060771] in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Information Technology-IT - DA of Alliance University, is a bonafide work during the academic year 2026-2027.

Prof. Ananthanagu U	Prof. Ananthanagu U	Dr. Shekhar R
Course Handling Faculty	Course Coordinator	Professor & HoD

Table of Contents

1. Introduction

1.1 Aim

1.2 Background

1.3 Problem Statement

1.4 Objectives

2. Methodology

2.1 Tools and Technologies Used

2.2 Approach / Development Process

3. Linux System Administration – Design and Implementation

3.1 System Architecture

3.2 Linux Environment Setup

3.3 User and Group Management

3.4 File and Directory Management

3.5 Package and Process Management

3.6 Storage and File System Configuration

3.7 Backup and Automation

3.8 Security and Access Control

4. Results and Testing

4.1 Implementation Results

4.2 Testing and Validation

5. Conclusion and Future Scope

References

Appendices

1. Introduction

1.1 Aim

The main aim of this mini project is to design, develop, and implement a **Linux Storage Monitoring System** that automatically monitors disk space, inode usage, and storage growth in a Linux environment.

The system uses Bash shell scripts to collect storage information, identify potential storage issues, generate alerts based on predefined thresholds, and maintain storage history logs. It also uses cron scheduling to automate monitoring tasks and generate storage reports at regular intervals.

1.2 Background

Linux is widely used in servers, cloud infrastructure, enterprise systems, networking, and software development. Proper storage management is an important part of Linux system administration because insufficient disk space or excessive inode usage can affect system performance and prevent applications from creating or storing files.

The **Linux Storage Monitoring System** was developed to simplify these tasks using Bash shell scripting and standard Linux utilities. It monitors disk space and inode usage, records storage measurements, checks warning and critical thresholds, and generates reports. Cron is used to schedule monitoring activities automatically.

1.3 Problem Statement

In Linux systems, storage space can become insufficient due to the accumulation of files, logs, and application data. High inode usage can also prevent new files from being created, even when disk space is available. Manual monitoring makes it difficult to identify these issues promptly.

Therefore, this project develops an automated Linux Storage Monitoring System to monitor disk space and inode usage, maintain storage records, generate threshold-based alerts, produce periodic reports, and perform basic backup and recovery checks.

1.4 Objectives

The main objectives of the Linux Storage Monitoring System are:

- To monitor disk space and inode usage using Linux commands and Bash scripts.

- To maintain storage history and track usage changes.
- To configure warning and critical storage thresholds.
- To generate alerts when storage usage exceeds the defined limits.
- To automate monitoring and report generation using cron scheduling.
- To generate readable storage reports.
- To perform basic backup and recovery checks.

2. Methodology

2.1 Tools and Technologies Used

2.1 Software Requirements

The Linux Storage Monitoring System was developed using Ubuntu Linux, Bash shell scripting, and standard Linux utilities to monitor storage, maintain logs, generate alerts, and produce reports.

- **Ubuntu Linux:** Operating system used for development and testing.
- **Bash Shell:** Used to create disk monitoring, inode monitoring, storage growth, alert, and report generation scripts.
- **Cron:** Automates monitoring every five minutes and report generation every fifteen minutes.
- **Linux Utilities:** Commands such as df, du, ls, find, ps, free, and systemctl support storage and system monitoring.
- **APT and Tree:** Used for package management and displaying the project directory structure.
- **Tar and Gzip:** Used to create and inspect compressed backups.
- **Text Editor:** Used to create and modify scripts and configuration files.
- **CSV and Log Files:** Used to store storage history, monitoring records, and administrator alerts.

2.2 Hardware Requirements

The project was implemented on a computer running Ubuntu Linux. The system requires sufficient resources to execute monitoring scripts, maintain logs, generate reports, and store backups.

General hardware requirements include:

- A laptop or desktop computer.

- A compatible processor capable of running Ubuntu Linux.
- Sufficient RAM to run the operating system and required utilities.
- Adequate storage space for scripts, logs, reports, historical data, and backups.

2.3 Other Tools and Resources

- **Linux Terminal:** Used to execute commands and run monitoring scripts.
- **System Monitoring Commands:** Used to inspect disk space, inode usage, processes, memory, and services.
- **Configuration File:** config/monitor.conf stores the monitoring path and warning and critical thresholds.
- **Cron Logs:** Used to inspect scheduled task execution.
- **Architecture Diagram:** Illustrates the relationship between monitoring scripts, scheduling, logs, reports, and backups.
- **Backup Verification Commands:** Used to inspect the backup archive and compare selected recovered files with their originals.

2.2 Approach / Development Process

The project was developed in stages, covering problem identification, requirement analysis, environment setup, implementation, automation, testing, and documentation.

Step 1: Problem Identification

The need for automated Linux storage monitoring was identified. The system was designed to monitor disk space and inode usage, record measurements, generate threshold-based alerts, and produce storage reports.

Step 2: Requirement Analysis

The project requirements included Ubuntu Linux, Bash, Cron, standard Linux utilities, the tree utility, and sufficient hardware and storage resources. Security requirements included appropriate file permissions, user and group management, and configurable monitoring thresholds.

Step 3: Linux Environment Setup

The project was implemented in an Ubuntu Linux environment. A project directory was created to organize scripts, configuration files, logs, historical data, reports, backups, tests, and documentation. Terminal commands were used to inspect filesystem usage and system services.

Step 4: System Configuration

The monitor.conf file was configured to define the monitoring path and warning and critical thresholds. User and group management was practised to demonstrate file ownership, permissions, and controlled access. System processes and services were also inspected using Linux administration commands.

Step 5: Implementation

The monitoring system was developed using Bash scripts, each responsible for a specific task:

- disk_monitor.sh – monitors disk space usage.
- inode_monitor.sh – checks inode usage.
- storage_growth.sh – records storage measurements and tracks changes.
- alert_monitor.sh – checks thresholds and records alerts.
- generate_report.sh – generates storage monitoring reports.

The scripts store their outputs in the project's data, log, and report directories.

Step 6: Automation

Cron was configured to run monitoring tasks every five minutes and report generation every fifteen minutes. The Cron service and scheduled task activity were inspected to verify the automation setup.

Step 7: Testing

The system was tested to verify disk and inode monitoring, CSV history recording, threshold-based alerts, report generation, and Cron configuration. The backup archive was inspected, and

selected recovered files were compared with their originals. User and group permissions were tested using a dedicated test user.

The tests demonstrated that the main monitoring scripts, alert checks, report generation, and permission demonstrations worked under the tested conditions. Backup verification was limited to archive inspection and selected file comparisons.

Step 8: Documentation

The final stage involved documenting the project structure, tools, configuration, scripts, testing procedures, and results. Screenshots were collected as implementation evidence, and a system architecture diagram was prepared to illustrate the relationships between the scripts, Cron scheduler, configuration file, storage history, alert logs, reports, and backup archive.

3. Linux System Administration – Design and Implementation

3.1 System Architecture

The Linux Storage Monitoring System is designed to monitor disk space, inode usage, and storage growth on a Linux system. It uses Bash scripts, configuration files, scheduled tasks, log files, and report generation to automate storage monitoring.

The system runs on Ubuntu Linux. The monitoring scripts collect storage information, compare usage against configured thresholds, record measurements, generate alerts when required, and create storage reports. A backup archive is also maintained to support project file recovery.

This figure 3.1 presents the system architecture and component relationships, providing visual evidence of the configuration or result described above.

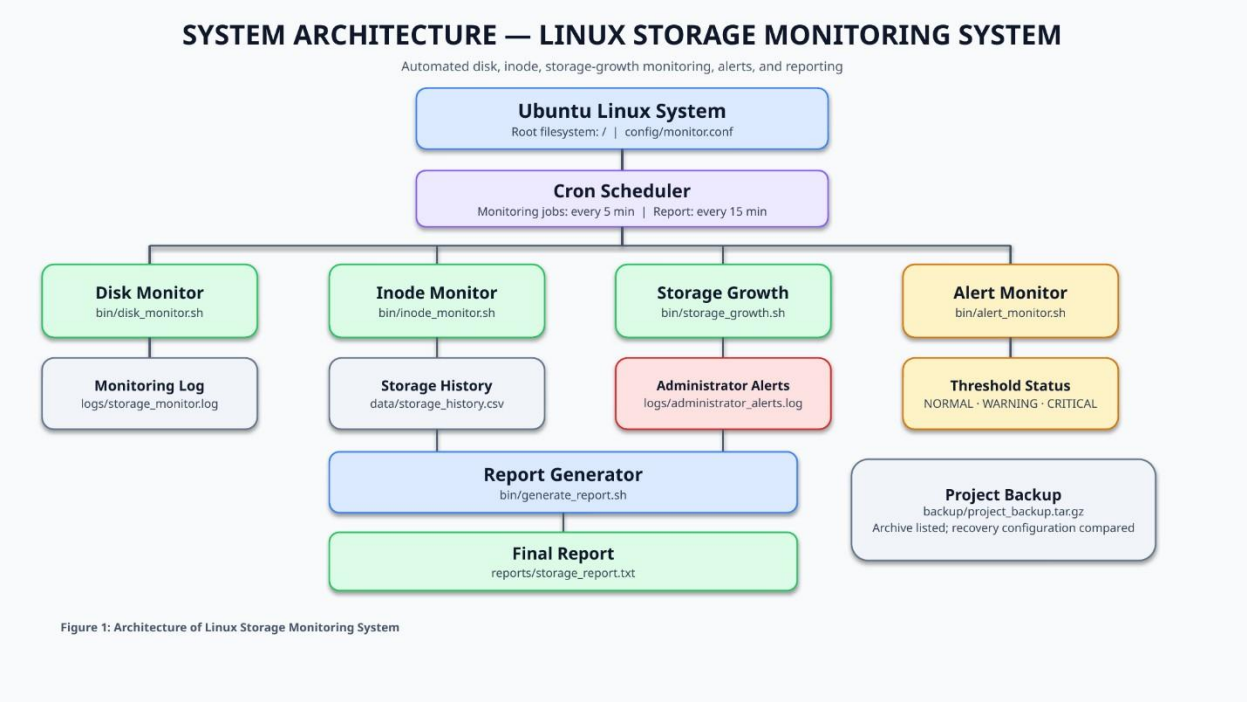


Figure 3.1: System Architecture of Linux Storage Monitoring System

Components and their roles

1. **Client/User**: The system administrator or user runs the scripts, checks storage reports, reviews logs, and manages project files.

2. Linux Server: Ubuntu Linux provides the operating environment for the monitoring scripts, system commands, scheduled tasks, and storage administration activities.

3. Storage: The root filesystem (/) is monitored for disk space and inode usage. Storage measurements are recorded in a CSV file to help track changes over time.

4. Network: Network connectivity belongs to the general Linux environment. The implemented storage monitoring functions primarily operate locally and do not require a network connection to collect disk usage information.

5. Applications/Services: Bash scripts perform disk monitoring, inode monitoring, storage-growth tracking, alert checking, and report generation. The Cron service schedules monitoring and report-generation tasks.

6. Backup System: A compressed backup archive, backup/project_backup.tar.gz, stores project files. The archive contents and selected recovered files were checked during testing.

7. Monitoring and Automation: The monitoring scripts use settings in config/monitor.conf. Measurements are saved in data/storage_history.csv, while monitoring and alert messages are recorded in the logs/ directory. Cron runs the scheduled tasks automatically, and the report generator creates reports/storage_report.txt.

3.2 Linux Environment Setup

3.2.1 Linux Distribution and Version

The project was developed using Ubuntu Linux 26.04.1 LTS. The system information was checked using the lsb_release -a command. The root filesystem is formatted with the ext4 filesystem and is mounted at /. The storage monitoring system uses Linux utilities to check disk space and inode utilization.

The figure 3.2 records the Ubuntu version and filesystem details and helps verify the corresponding implementation step.

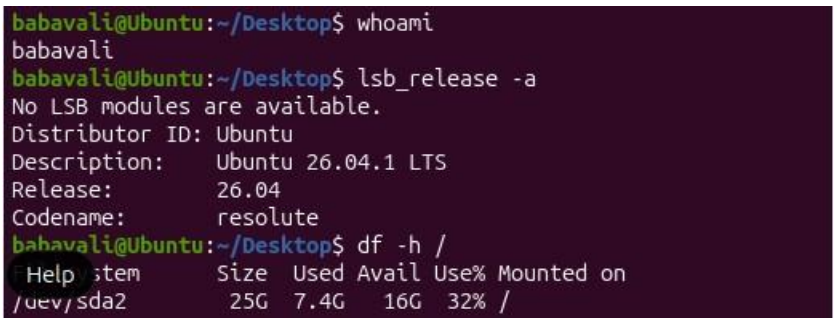
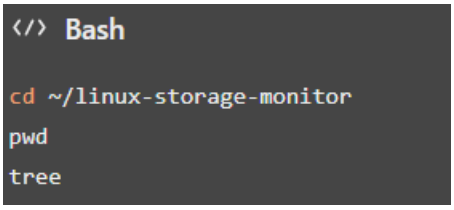


Figure 3.2: Ubuntu version and filesystem details

3.2.2 Installation Process

The project directory was created in the user's home directory using the mkdir command. The main directory, linux-storage-monitor, contains separate folders for monitoring scripts, configuration files, logs, storage history, reports, tests, documentation, and backups. The tree utility was used to display and verify the project structure.

BASH



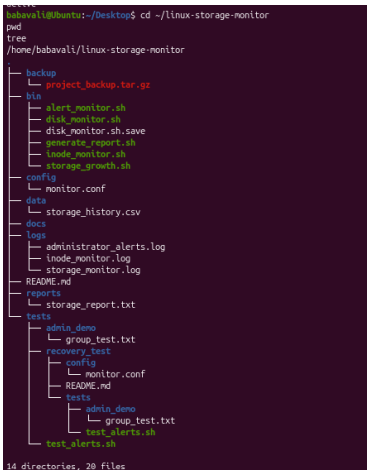


Figure 3.3:Project directory structure and installation

The figure 3.3 image documents project directory structure and installation and supports the explanation in this subsection.

3.2.3 Hostname Configuration

The hostname is used to identify the Linux system. The hostname and operating system details were checked using Linux system commands. The terminal prompt identifies the system as Ubuntu and displays the current account as babavali. These details help identify the environment in which the storage monitoring scripts are executed.

```bash
hostname
hostnamectl
whoami
```

The figure 3.4 below illustrates project directory structure and installation as observed during project setup and validation.

```
babavali@Ubuntu:~/linux-storage-monitor$ hostname
hostnamectl
whoami
Ubuntu
 Static hostname: Ubuntu
       Icon name: computer-vm
         Chassis: vm
      Machine ID: c0cc61162ce648f58e8b268ae7337537
         Boot ID: 60256a01039849729b4c63d4ca75d313
  Virtualization: oracle
Operating System: Ubuntu 26.04.1 LTS
          Kernel: Linux 7.0.0-38-generic
    Architecture: x86-64
 Hardware Vendor: innotek GmbH
  Hardware Model: VirtualBox
Hardware Version: 1.2
Firmware Version: VirtualBox
   Firmware Date: Fri 2006-12-01
    Firmware Age: 19y 10month 1w 4d
babavali
babavali@Ubuntu:~/linux-storage-monitor$
```

Figure 3.4: Project directory structure and installation

3.2.4 Network Configuration

The network configuration of the Linux environment was inspected using standard networking commands. The commands display the available network interfaces, assigned IP addresses, and routing information. The storage monitoring scripts operate locally and do not require network connectivity to collect disk usage and inode information.

The figure 3.5 records the project directory structure and installation and helps verify the corresponding implementation step .

```bash
ip address
ip route
systemctl is-active NetworkManager
```

```
babavali@Ubuntu:~/Desktop$ ip address
ip route
systemctl is-active NetworkManager
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:86:f7:64 brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute enp0s3
       valid_lft 86011sec preferred_lft 86011sec
    inet6 fd17:625c:f037:2:a00:27ff:fe86:f764/64 scope global dynamic noprefixroute
       valid_lft 86185sec preferred_lft 14185sec
    inet6 fe80::a00:27ff:fe86:f764/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
default via 10.0.2.2 dev enp0s3 proto dhcp src 10.0.2.15 metric 100
10.0.2.0/24 dev enp0s3 proto kernel scope link src 10.0.2.15 metric 100
active
```

Figure 3.5: Project directory structure and installation

3.2.5 User Accounts

User and group management was demonstrated as part of Linux system administration. The project was executed using the babavali account. A test user named storagetest and a group named storageproject were used to demonstrate group membership and access permissions. File ownership and permissions were checked using a demonstration directory.

The figure 3.6 below illustrates project directory structure and installation as observed during project setup and validation

```bash
whoami
id storagetest
getent group storageproject
ls -ld /tmp/storageproject_demo
ls -l /tmp/storageproject_demo
```

```
babavali@Ubuntu:~/linux-storage-monitor$ whoami
id storagetest
getent group storageproject
ls -ld /tmp/storageproject_demo
ls -l /tmp/storageproject_demo
babavali
uid=1009(storagetest) gid=1009(storagetest) groups=1009(storagetest),100(users),1008(storageproject)
storageproject:x:1008:storagetest
drwxrws--- 2 root storageproject 60 Oct 12 11:54 /tmp/storageproject_demo
ls: cannot open directory '/tmp/storageproject_demo': Permission denied
```

Figure 3.6: Project directory structure and installation

3.2.6 Required Packages

The project uses Bash and standard Linux utilities for storage monitoring, file management, and system administration. The tree utility is used to display the project directory structure. The tar and gzip utilities are used to create and inspect the compressed backup archive. These utilities support project organization, monitoring, and backup operations.

```
Bash

bash --version
tree --version
tar --version
gzip --version
dpkg -s tree | grep -E "Status|Version"
```

```
babavali@Ubuntu:~/linux-storage-monitor$ bash --version
tree --version
tar --version
gzip --version
dpkg -s tree | grep -E "Status|Version"
GNU bash, version 5.3.9(1)-release (x86_64-pc-linux-gnu)
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>

This is free software; you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
tree v2.3.1 © 1996 - 2026 by Steve Baker, Thomas Moore, Francesc Rocher, Florian Sesser, Kyosuke Tokoro
tar (GNU tar) 1.35
Copyright (C) 2023 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <https://gnu.org/licenses/gpl.html>.
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

Written by John Gilmore and Jay Fenlason.
gzip 1.14
Copyright (C) 2025 Free Software Foundation, Inc.
Copyright (C) 1993 Jean-loup Gailly.
This is free software.  You may redistribute copies of it under the terms of
the GNU General Public License <https://www.gnu.org/licenses/gpl.html>.
There is NO WARRANTY, to the extent permitted by law.

Written by Jean-loup Gailly.
Status: install ok installed
Version: 2.3.1-1
```

Figure 3.7: required packages

3.2.7 System Services

The Cron service was checked to verify that the Linux task scheduler was active. Cron was configured to execute the disk monitoring, inode monitoring, storage growth, and alert monitoring

scripts every five minutes. The report generation script was scheduled to run every fifteen minutes. The configured tasks were verified using the crontab -l command.

The figure 3.8 records the project directory structure and installation and helps verify the corresponding implementation step

```bash
systemctl status cron --no-pager
systemctl is-active cron
crontab -l
```

```
babavali@Ubuntu:~/linux-storage-monitor$ systemctl status cron --no-pager
systemctl is-active cron
crontab -l
● cron.service - Regular background program processing daemon
     Loaded: loaded (/usr/lib/systemd/system/cron.service; enabled; preset: enabled)
     Active: active (running) since Mon 2026-10-12 11:29:20 UTC; 27min ago
 Invocation: cf6d9a6862f846c98f904a5ad4658d3a
       Docs: man:cron(8)
   Main PID: 1056 (cron)
      Tasks: 1 (limit: 5138)
     Memory: 860K (peak: 16.1M)
        CPU: 14.048s
     CGroup: /system.slice/cron.service
             └─1056 /usr/sbin/cron -f -P

Oct 12 11:55:01 Ubuntu CRON[10302]: (babavali) CMD (/home/babavali/linux-storage-monitor/bin/storage_growth.sh)
Oct 12 11:55:01 Ubuntu CRON[10306]: (babavali) CMD (/home/babavali/linux-storage-monitor/bin/inode_monitor.sh)
Oct 12 11:55:01 Ubuntu CRON[10294]: (CRON) info (No MTA installed, discarding output)
Oct 12 11:55:01 Ubuntu CRON[10294]: pam_unix(cron:session): session closed for user babavali
Oct 12 11:55:01 Ubuntu CRON[10293]: (CRON) info (No MTA installed, discarding output)
Oct 12 11:55:01 Ubuntu CRON[10293]: pam_unix(cron:session): session closed for user babavali
Oct 12 11:55:01 Ubuntu CRON[10291]: (CRON) info (No MTA installed, discarding output)
Oct 12 11:55:01 Ubuntu CRON[10291]: pam_unix(cron:session): session closed for user babavali
Oct 12 11:55:01 Ubuntu CRON[10292]: (CRON) info (No MTA installed, discarding output)
Oct 12 11:55:01 Ubuntu CRON[10292]: pam_unix(cron:session): session closed for user babavali
active
*/5 * * * * /home/babavali/linux-storage-monitor/bin/disk_monitor.sh
*/5 * * * * /home/babavali/linux-storage-monitor/bin/inode_monitor.sh
*/5 * * * * /home/babavali/linux-storage-monitor/bin/storage_growth.sh
*/5 * * * * /home/babavali/linux-storage-monitor/bin/alert_monitor.sh
*/15 * * * * /home/babavali/linux-storage-monitor/bin/generate_report.sh
babavali@Ubuntu:~/linux-storage-monitor$
```

Figure 3.8: Project directory structure and installation

3.3 User and Group Management

3.3.1 User Creation

A test user named storagetest was created in the Ubuntu Linux environment to demonstrate user account management. The account was used to test group membership and file access permissions. The id and getent passwd commands were used to verify the user account information.

The figure 3.9 documents Ubuntu version and filesystem details and supports the explanation in this subsection.

```bash
id storagetest
getent passwd storagetest
```

```
babavali@Ubuntu:~/linux-storage-monitor$ id storagetest
getent passwd storagetest
uid=1009(storagetest) gid=1009(storagetest) groups=1009(storagetest),100(users),1008(storageproject)
storagetest:x:1009:1009:,1,12345,123456789:/home/storagetest:/bin/bash
babavali@Ubuntu:~/linux-storage-monitor$
```

Figure 3.9: Ubuntu version and filesystem details

3.3.2 Group Creation

A group named storageproject was created to demonstrate group management in Linux. Groups help administrators manage permissions and provide access to files and directories for multiple users. The getent group command was used to verify the group and its membership.

The figure 3.10 illustrates test user account configuration as observed during project setup and validation.

```bash
getent group storageproject
```

```
babavali@Ubuntu:~/linux-storage-monitor$ getent group storageproject
storageproject:x:1008:storagetest
```

Figure 3.10: Test user account configuration

3.3.3 User-Group Assignment

The existing user storagetest was assigned to the storageproject group. This assignment demonstrates how Linux group membership can be used to manage access to project-related files and directories. The id command was used to verify the user's group membership.

The figure 3.11 records the test user account configuration and helps verify the corresponding implementation step.

```bash
id storagetest
getent group storageproject
```

```
babavali@Ubuntu:~/linux-storage-monitor$ id storagetest
getent group storageproject
uid=1009(storagetest) gid=1009(storagetest) groups=1009(storagetest),100(users),1008(storageproject)
storageproject:x:1008:storagetest
babavali@Ubuntu:~/linux-storage-monitor$
```

Figure 3.11**:** Test user account configuration

3.3.4 Password Management

Password management was demonstrated during the creation and configuration of the test user account. Linux provides administrative commands for setting and updating user passwords. Passwords are managed securely by the operating system and are not displayed as plain text during normal account operations.

The figure 3.12 documents test user account configuration and supports the explanation in this subsection.

```bash
sudo passwd storagetest
```

```
babavali@Ubuntu:~/linux-storage-monitor$ sudo passwd storagetest
[sudo: authenticate] Password:
New password:
Retype new password:
passwd: password updated successfully
```

Figure 3.12:Test user account configuration

3.3.5 File Ownership

File ownership was configured to demonstrate Linux ownership management. The demonstration directory and file were assigned to the root user and the storageproject group. The chown and ls -l commands were used to configure and verify ownership.

The figure 3.13 documents test user account configuration and supports the explanation in this subsection.

```bash
ls -ld /tmp/storageproject_demo
ls -l /tmp/storageproject_demo
```

```
babavali@Ubuntu:~/linux-storage-monitor$ ls -ld /tmp/storageproject_demo
ls -l /tmp/storageproject_demo
drwxrws--- 2 root storageproject 60 Oct 12 11:54 /tmp/storageproject_demo
ls: cannot open directory '/tmp/storageproject_demo': Permission denied
babavali@Ubuntu:~/linux-storage-monitor$
```

Figure 3.13:Test user account configuration

3.3.6 File Permissions

Linux file permissions control the operations that users can perform on files and directories. The demonstration directory was configured with permission 2770, which grants access to the owner and group and enables the set-group-ID bit. The demonstration file was assigned permission 640, which grants read and write access to the owner, read access to the group, and no access to other users.

The figure 3.14 records the test user account configuration and helps verify the corresponding implementation step.

```bash
ls -ld /tmp/storageproject_demo
ls -l /tmp/storageproject_demo
```

```
babavali@Ubuntu:~/linux-storage-monitor$ ls -ld /tmp/storageproject_demo
ls -l /tmp/storageproject_demo
drwxrws--- 2 root storageproject 60 Oct 12 11:54 /tmp/storageproject_demo
ls: cannot open directory '/tmp/storageproject_demo': Permission denied
babavali@Ubuntu:~/linux-storage-monitor$
```

Figure 3.14: Test user account configuration

3.3.7 Access Control

Access control was tested using the storagetest account. The test user was used to access a demonstration file belonging to the storageproject group. The sudo -u command allows an administrator to execute a command as another user. This test demonstrates how Linux permissions determine whether a user can read a file.

The figure 3.15 documents test user account configuration and supports the explanation in this subsection.

```
</> Bash

sudo -u storagetest cat /tmp/storageproject_demo/group_test.txt
```

```
babavali@Ubuntu:~/linux-storage-monitor$ sudo -u storagetest cat /tmp/storageproject_demo/group_test.txt
Linux storage permission demonstration
babavali@Ubuntu:~/linux-storage-monitor$
```

Figure 3.15: Test user account configuration

3.4. File and Directory Management

File and directory management is an important part of the Linux Storage Monitoring System. The project uses a structured directory layout to store monitoring scripts, configuration files, log files, storage history, reports, backups, and test files. This organization makes the project easier to manage, maintain, and troubleshoot.

3.4.1 Directory Structure

The project directory linux-storage-monitor contains separate folders for scripts, configuration, logs, data, reports, backups, tests, and documentation. This structure keeps the project files organized and makes them easy to locate.

3.4.2 File Creation and Management

The project uses Bash scripts to monitor disk usage, inode usage, storage growth, and storage alerts. Configuration settings are stored in monitor.conf. Monitoring information is recorded in log files and storage_history.csv, while the generated storage summary is saved in storage_report.txt.

3.4.3 File Permissions and Ownership

Linux file permissions are used to control access to files and directories. The project scripts have permissions that allow them to be executed as required. The project also demonstrates ownership and group permissions using the storageproject group and the storagetest user. These permissions help control access to shared files.

3.4.4 Symbolic Links

A symbolic link was created to demonstrate how Linux can provide an alternative path to an existing file. The project includes group_test_link.txt, which points to group_test.txt. Symbolic links help users access files through different path names without creating another copy of the original file.

3.4.5 Disk Usage Monitoring

Disk usage is monitored using Linux commands and the project's monitoring scripts. The df command displays filesystem space usage, while du displays the disk space used by files and directories. The project also checks inode usage to identify possible inode exhaustion. Warning and critical thresholds are defined in the configuration file to help identify storage usage that requires attention.

3.4.6 File Searching and Log Filtering

Linux utilities such as find and grep are used to locate project files and search for specific configuration settings or log entries. The tail command helps display recent log messages. These utilities make it easier to review monitoring activity, investigate alerts, and troubleshoot storagerelated issues.

Result: The project demonstrates organized file management, Linux permissions and ownership, symbolic links, disk and inode usage monitoring, and basic file searching and log analysis.

3.5 Package and Process Management

Package and process management are important aspects of Linux system administration. Package management is used to install, update, and remove software, while process management is used to monitor running programs and manage system resources. In the Linux Storage Monitoring System, these functions help maintain the required tools and monitor system activity.

3.5.1 Package Installation and Removal

The Advanced Package Tool (APT) is used to install and manage software packages in Ubuntu Linux. The tree utility was installed to display the project directory structure. Installed packages can be checked using package management commands.

The figure 3.16 below illustrates Ubuntu version and filesystem details as observed during project setup and validation.

Commands:

```bash
sudo apt update
sudo apt install tree
tree --version
dpkg -s tree
```

```
babavali@Ubuntu:~/linux-storage-monitor$ sudo apt update
sudo apt install tree
tree --version
dpkg -s tree
Get:1 http://security.ubuntu.com/ubuntu resolute-security InRelease [137 kB]
Hit:2 http://in.archive.ubuntu.com/ubuntu resolute InRelease
Get:3 http://in.archive.ubuntu.com/ubuntu resolute-updates InRelease [137 kB]
Get:4 http://security.ubuntu.com/ubuntu resolute-security/main amd64 Packages [600 kB]
Get:5 http://in.archive.ubuntu.com/ubuntu resolute-backports InRelease [137 kB]
Get:6 http://security.ubuntu.com/ubuntu resolute-security/main amd64 Components [46.7 kB]
Get:7 http://security.ubuntu.com/ubuntu resolute-security/universe amd64 Components [55.5 kB]
Get:8 http://in.archive.ubuntu.com/ubuntu resolute-updates/main amd64 Packages [734 kB]
Get:9 http://in.archive.ubuntu.com/ubuntu resolute-updates/main amd64 Components [110 kB]
Get:10 http://in.archive.ubuntu.com/ubuntu resolute-updates/universe amd64 Packages [323 kB]
Get:11 http://in.archive.ubuntu.com/ubuntu resolute-updates/universe amd64 Components [201 kB]
Get:12 http://in.archive.ubuntu.com/ubuntu resolute-backports/universe amd64 Components [1,056 B]
Fetched 2,483 kB in 2s (1,171 kB/s)
83 packages can be upgraded. Run 'apt list --upgradable' to see them.
tree is already the newest version (2.3.1-1).
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 83
tree v2.3.1 © 1996 - 2026 by Steve Baker, Thomas Moore, Francesc Rocher, Florian Sesser, Kyosuke Tokoro
Package: tree
Status: install ok installed
Priority: optional
Section: utils
Installed-Size: 122
Maintainer: Ubuntu Developers <ubuntu-devel-discuss@lists.ubuntu.com>
Architecture: amd64
Version: 2.3.1-1
Depends: libc6 (>= 2.38)
Description: displays an indented directory tree, in color
 Tree is a recursive directory listing command that produces a depth indented
 listing of files, which is colorized ala dircolors if the LS_COLORS environment
 variable is set and output is to tty.
Original-Maintainer: Florian Ernst <florian@debian.org>
Homepage: http://oldmanprogrammer.net/source.php?dir=projects/tree
```

Figure 3.16:Ubuntu version and filesystem details

3.5.2 Dependency Management

Dependency management ensures that the software and utilities required by the project are available. The monitoring system uses Bash scripts and standard Linux commands to check storage usage, process information, and system resources. APT helps install required packages and manage their dependencies.

The figure 3.17 records the required Linux utilities and helps verify the corresponding implementation step.

```bash
bash --version
df --version
du --version
grep --version
dpkg -s tree
```

```
babavali@Ubuntu:~/linux-storage-monitor$ bash --version
df --version
du --version
grep --version
dpkg -s tree
GNU bash, version 5.3.9(1)-release (x86_64-pc-linux-gnu)
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>

This is free software; you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
df (GNU coreutils) 9.7
Packaged by Ubuntu (9.7-3ubuntu2.1)
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <https://gnu.org/licenses/gpl.html>.
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

Written by Torbjörn Granlund, David MacKenzie, and Paul Eggert.
du (uutils coreutils) 0.8.0
grep (GNU grep) 3.12
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <https://gnu.org/licenses/gpl.html>.
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

Written by Mike Haertel and others; see
<https://git.savannah.gnu.org/cgit/grep.git/tree/AUTHORS>.

grep -P uses PCRE2 10.46 2025-08-27
Package: tree
Status: install ok installed
Priority: optional
Section: utils
Installed-Size: 122
Maintainer: Ubuntu Developers <ubuntu-devel-discuss@lists.ubuntu.com>
Architecture: amd64
Version: 2.3.1-1
Depends: libc6 (>= 2.38)
Description: displays an indented directory tree, in color
 Tree is a recursive directory listing command that produces a depth indented
 listing of files, which is colorized ala dircolors if the LS_COLORS environment
 variable is set and output is to tty.
Original-Maintainer: Florian Ernst <florian@debian.org>
Homepage: http://oldmanprogrammer.net/source.php?dir=projects/tree
```

Figure 3.17:Required Linux utilities

3.5.3 Software Updates

Software updates help maintain system stability, improve security, and fix known issues. Ubuntu provides APT commands to refresh package information and install available updates.

The figure 3.18 documents system memory and process status and supports the explanation in this subsection.

```bash
sudo apt update
apt list --upgradable
```

```
babavali@Ubuntu:~/linux-storage-monitor$ sudo apt update
apt list --upgradable
Hit:1 http://in.archive.ubuntu.com/ubuntu resolute InRelease
Hit:2 http://security.ubuntu.com/ubuntu resolute-security InRelease
Hit:3 http://in.archive.ubuntu.com/ubuntu resolute-updates InRelease
Hit:4 http://in.archive.ubuntu.com/ubuntu resolute-backports InRelease
83 packages can be upgraded. Run 'apt list --upgradable' to see them.
alsa-ucm-conf/resolute-updates 1.2.15.3-1ubuntu1.7 all [upgradable from: 1.2.15.3-1ubuntu1.5]
apache2-bin/resolute-updates,resolute-security 2.4.66-2ubuntu2.5 amd64 [upgradable from: 2.4.66-2ubuntu2.4]
apache2-data/resolute-updates,resolute-security 2.4.66-2ubuntu2.5 all [upgradable from: 2.4.66-2ubuntu2.4]
apache2-utils/resolute-updates,resolute-security 2.4.66-2ubuntu2.5 amd64 [upgradable from: 2.4.66-2ubuntu2.4]
apache2/resolute-updates,resolute-security 2.4.66-2ubuntu2.5 amd64 [upgradable from: 2.4.66-2ubuntu2.4]
bluez-cups/resolute-updates,resolute-security 5.85-4ubuntu0.3 amd64 [upgradable from: 5.85-4ubuntu0.1]
bluez-obexd/resolute-updates,resolute-security 5.85-4ubuntu0.3 amd64 [upgradable from: 5.85-4ubuntu0.1]
bluez/resolute-updates,resolute-security 5.85-4ubuntu0.3 amd64 [upgradable from: 5.85-4ubuntu0.1]
dmidecode/resolute-updates 3.6-2ubuntu1 amd64 [upgradable from: 3.6-2build1]
dnsmasq-base/resolute-updates 2.92-1ubuntu0.5 amd64 [upgradable from: 2.92-1ubuntu0.4]
firefox/resolute-updates 1:1snap1-0ubuntu9.1 amd64 [upgradable from: 1:1snap1-0ubuntu8]
gir1.2-mutter-18/resolute-updates 50.1-0ubuntu2.4 amd64 [upgradable from: 50.1-0ubuntu2.2]
glycin-loaders/resolute-updates 2.1.5+ds-0ubuntu0.2 amd64 [upgradable from: 2.1.1+ds-0ubuntu1]
glycin-thumbnailers/resolute-updates 2.1.5+ds-0ubuntu0.2 amd64 [upgradable from: 2.1.1+ds-0ubuntu1]
gnome-control-center-data/resolute-updates 1:50.3-0ubuntu0.2 all [upgradable from: 1:50.3-0ubuntu0.1]
gnome-control-center-faces/resolute-updates 1:50.3-0ubuntu0.2 all [upgradable from: 1:50.3-0ubuntu0.1]
gnome-control-center/resolute-updates 1:50.3-0ubuntu0.2 amd64 [upgradable from: 1:50.3-0ubuntu0.1]
gnome-shell-common/resolute-updates 50.1-0ubuntu1.3 all [upgradable from: 50.1-0ubuntu1.2]
gnome-shell-ubuntu-extensions/resolute-updates 50.26.04.8ubuntu all [upgradable from: 50.26.04.7ubuntu]
gnome-shell/resolute-updates 50.1-0ubuntu1.3 amd64 [upgradable from: 50.1-0ubuntu1.2]
gstreamer1.0-pipewire/resolute-updates 1.6.2-1ubuntu1.2 amd64 [upgradable from: 1.6.2-1ubuntu1.1]
libarchive13t64/resolute-updates,resolute-security 3.8.5-1ubuntu2.3 amd64 [upgradable from: 3.8.5-1ubuntu2.2]
libaudit-common/resolute-updates 1:4.1.2-1ubuntu0.1 all [upgradable from: 1:4.1.2-1build1]
libaudit1/resolute-updates 1:4.1.2-1ubuntu0.1 amd64 [upgradable from: 1:4.1.2-1build1]
libbluetooth3/resolute-updates,resolute-security 5.85-4ubuntu0.3 amd64 [upgradable from: 5.85-4ubuntu0.1]
libflashrom1/resolute-updates 1.6.0-2ubuntu1.1 amd64 [upgradable from: 1.6.0-2ubuntu1]
libglycin-2-0/resolute-updates 2.1.5+ds-0ubuntu0.2 amd64 [upgradable from: 2.1.1+ds-0ubuntu1]
libglycin-gtk4-2-0/resolute-updates 2.1.5+ds-0ubuntu0.2 amd64 [upgradable from: 2.1.1+ds-0ubuntu1]
libipa-hbac0t64/resolute-updates 2.12.0-1ubuntu5.5 amd64 [upgradable from: 2.12.0-1ubuntu5.4]
libmalcontent-0-0/resolute-updates 0.14.0-0ubuntu1.2 amd64 [upgradable from: 0.14.0-0ubuntu1.1]
libmalcontent-common/resolute-updates 0.14.0-0ubuntu1.2 all [upgradable from: 0.14.0-0ubuntu1.1]
libmp3lame0/resolute-updates 3.101~svn6525+dfsg-2ubuntu0.1 amd64 [upgradable from: 3.101~svn6525+dfsg-2]
libmutter-18-0/resolute-updates 50.1-0ubuntu2.4 amd64 [upgradable from: 50.1-0ubuntu2.2]
libnautilus-extension4/resolute-updates 1:50.2.2-0ubuntu0.2 amd64 [upgradable from: 1:50.2.2-0ubuntu0.1]
libnetplan1/resolute-updates 1.2-1ubuntu5.1 amd64 [upgradable from: 1.2-1ubuntu5]
libnss-sss/resolute-updates 2.12.0-1ubuntu5.5 amd64 [upgradable from: 2.12.0-1ubuntu5.4]
libpam-sss/resolute-updates 2.12.0-1ubuntu5.5 amd64 [upgradable from: 2.12.0-1ubuntu5.4]
```

Figure 3.18: System memory and process status

3.5.4 Process Monitoring

Process monitoring helps identify programs currently running on the Linux system. Commands such as ps, top, and free are used to observe processes, CPU activity, memory usage, and overall system performance.

The figure 3.19 illustrates system memory and process status as observed during project setup and validation.

```
Bash

ps aux | head -n 15
top -b -n 1 | head -n 12
free -h
```

```
babavali@Ubuntu:~/linux-storage-monitor$ ps aux | head -n 15
top -b -n 1 | head -n 12
free -h
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  1.9  0.2  26324 17232 ?        Ss   11:27   1:24 /usr/lib/systemd/systemd --switched-root --system --deserialize=51
root           2  0.0  0.0      0     0 ?        S    11:27   0:00 [kthreadd]
root           3  0.0  0.0      0     0 ?        S    11:27   0:00 [pool_workqueue_release]
root           4  0.0  0.0      0     0 ?        I<   11:27   0:00 [kworker/R-rcu_gp]
root           5  0.0  0.0      0     0 ?        I<   11:27   0:00 [kworker/R-sync_wq]
root           6  0.0  0.0      0     0 ?        I<   11:27   0:00 [kworker/R-kvfree_rcu_reclaim]
root           7  0.0  0.0      0     0 ?        I<   11:27   0:00 [kworker/R-slub_flushwq]
root           8  0.0  0.0      0     0 ?        I<   11:27   0:00 [kworker/R-netns]
root          12  0.0  0.0      0     0 ?        I    11:27   0:00 [kworker/u24:0]
root          13  0.0  0.0      0     0 ?        I<   11:27   0:00 [kworker/R-mm_percpu_wq]
root          14  0.2  0.0      0     0 ?        S    11:27   0:09 [ksoftirqd/0]
root          15  0.2  0.0      0     0 ?        I    11:27   0:09 [rcu_preempt]
root          16  0.0  0.0      0     0 ?        S    11:27   0:00 [rcu_exp_par_gp_kthread_worker/0]
root          17  0.0  0.0      0     0 ?        S    11:27   0:03 [rcu_exp_gp_kthread_worker]
top - 12:38:56 up  1:11,  1 user,  load average: 0.42, 0.33, 0.35
Tasks: 273 total,   1 running, 270 sleeping,   2 stopped,   0 zombie
%Cpu(s):  2.4 us,  2.4 sy,  0.0 ni, 92.7 id,  1.2 wa,  0.0 hi,  1.2 si,  0.0 st
MiB Mem :   6343.4 total,    891.7 free,   2271.6 used,   3226.2 buff/cache
MiB Swap:      0.0 total,      0.0 free,      0.0 used.   4071.8 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
      1 root      20   0   26324  17232  11716 S   0.0   0.3   1:24.72 systemd
      2 root      20   0       0      0      0 S   0.0   0.0   0:00.82 kthreadd
      3 root      20   0       0      0      0 S   0.0   0.0   0:00.00 pool_wo+
      4 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker+
      5 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker+
               total        used        free      shared  buff/cache   available
Mem:           6.2Gi       2.2Gi       891Mi        52Mi       3.2Gi       4.0Gi
Swap:             0B          0B          0B
```

Figure 3.19: System memory and process status

3.5.5 Process Termination

Linux provides the kill command to terminate a process when necessary. A process ID (PID) identifies each running process. Before terminating a process, its PID should be checked carefully to avoid stopping an important system service.

The figure 3.20 records the Cron service and scheduled automation and helps verify the corresponding implementation step.

```bash
sleep 300 &
ps -ef | grep '[s]leep 300'
```

```
babavali@Ubuntu:~/linux-storage-monitor$ sleep 300 &
ps -ef | grep '[s]leep 300'
[2] 12234
babavali   12234    9421 99 12:39 pts/0    00:00:00 sleep 300
```

Figure 3.20: Cron service and scheduled automation

3.5.6 Service Management

Linux services can be managed using systemctl. The Cron service is important to this project because it runs scheduled monitoring tasks automatically. The service status can be checked to verify that it is active.

The figure 3.21 documents Cron service and scheduled automation and supports the explanation in this subsection.

```bash
systemctl status cron --no-pager
systemctl is-active cron
crontab -l
```

```
babavali@Ubuntu:~/linux-storage-monitor$ systemctl status cron --no-pager
systemctl is-active cron
crontab -l
● cron.service - Regular background program processing daemon
     Loaded: loaded (/usr/lib/systemd/system/cron.service; enabled; preset: enabled)
     Active: active (running) since Mon 2026-10-12 11:29:20 UTC; 1h 13min ago
 Invocation: cf6d9a6862f846c98f904a5ad4658d3a
       Docs: man:cron(8)
   Main PID: 1056 (cron)
      Tasks: 1 (limit: 5138)
     Memory: 872K (peak: 16.8M)
        CPU: 35.923s
     CGroup: /system.slice/cron.service
             └─1056 /usr/sbin/cron -f -P

Oct 12 12:40:01 Ubuntu CRON[12256]: (babavali) CMD (/home/babavali/linux-storage-monitor/bin/disk_monitor.sh)
Oct 12 12:40:01 Ubuntu CRON[12257]: (babavali) CMD (/home/babavali/linux-storage-monitor/bin/alert_monitor.sh)
Oct 12 12:40:01 Ubuntu CRON[12247]: (CRON) info (No MTA installed, discarding output)
Oct 12 12:40:01 Ubuntu CRON[12247]: pam_unix(cron:session): session closed for user babavali
Oct 12 12:40:02 Ubuntu CRON[12245]: (CRON) info (No MTA installed, discarding output)
Oct 12 12:40:02 Ubuntu CRON[12248]: (CRON) info (No MTA installed, discarding output)
Oct 12 12:40:02 Ubuntu CRON[12245]: pam_unix(cron:session): session closed for user babavali
Oct 12 12:40:02 Ubuntu CRON[12248]: pam_unix(cron:session): session closed for user babavali
Oct 12 12:40:02 Ubuntu CRON[12246]: (CRON) info (No MTA installed, discarding output)
Oct 12 12:40:02 Ubuntu CRON[12246]: pam_unix(cron:session): session closed for user babavali
active
*/5 * * * * /home/babavali/linux-storage-monitor/bin/disk_monitor.sh
*/5 * * * * /home/babavali/linux-storage-monitor/bin/inode_monitor.sh
*/5 * * * * /home/babavali/linux-storage-monitor/bin/storage_growth.sh
*/5 * * * * /home/babavali/linux-storage-monitor/bin/alert_monitor.sh
*/15 * * * * /home/babavali/linux-storage-monitor/bin/generate_report.sh
babavali@Ubuntu:~/linux-storage-monitor$
```

Figure 3.21: Cron service and scheduled automation

3.5.7 Resource Monitoring

Resource monitoring helps observe disk space, inode availability, and memory usage. The project uses Linux commands and monitoring scripts to identify storage usage and record monitoring information. Warning and critical thresholds are defined in the configuration file.

The figure 3.22 illustrates Cron service and scheduled automation as observed during project setup and validation.

```bash
df -h /
df -i /
free -h
du -sh ~/linux-storage-monitor
```

```
babavali@Ubuntu:~/linux-storage-monitor$ df -h /
df -i /
free -h
du -sh ~/linux-storage-monitor
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda2        25G  7.5G   16G  33% /
Filesystem      Inodes  IUsed   IFree IUse% Mounted on
/dev/sda2      1638400 217485 1420915   14% /
               total        used        free      shared  buff/cache   available
Mem:           6.2Gi       2.2Gi       890Mi        52Mi       3.2Gi       4.0Gi
Swap:            0B          0B          0B
140K    /home/babavali/linux-storage-monitor
babavali@Ubuntu:~/linux-storage-monitor$
```

Figure 3.22: Cron service and scheduled automation

3.6 Storage and File System Configuration

Storage and file system configuration are important parts of Linux system administration. They help administrators understand disk partitions, file systems, mount points, and available storage space. The Linux Storage Monitoring System uses Linux commands to monitor disk space and inode usage and identify storage conditions that may require attention.

3.6.1 Disk and Partition Configuration

The df and lsblk commands are used to view filesystem usage and available block devices. The project monitors the root filesystem (/) using the configured MONITOR_PATH=/ setting.

The figure 3.23 **r**ecords the disk space monitoring and helps verify the corresponding implementation step.

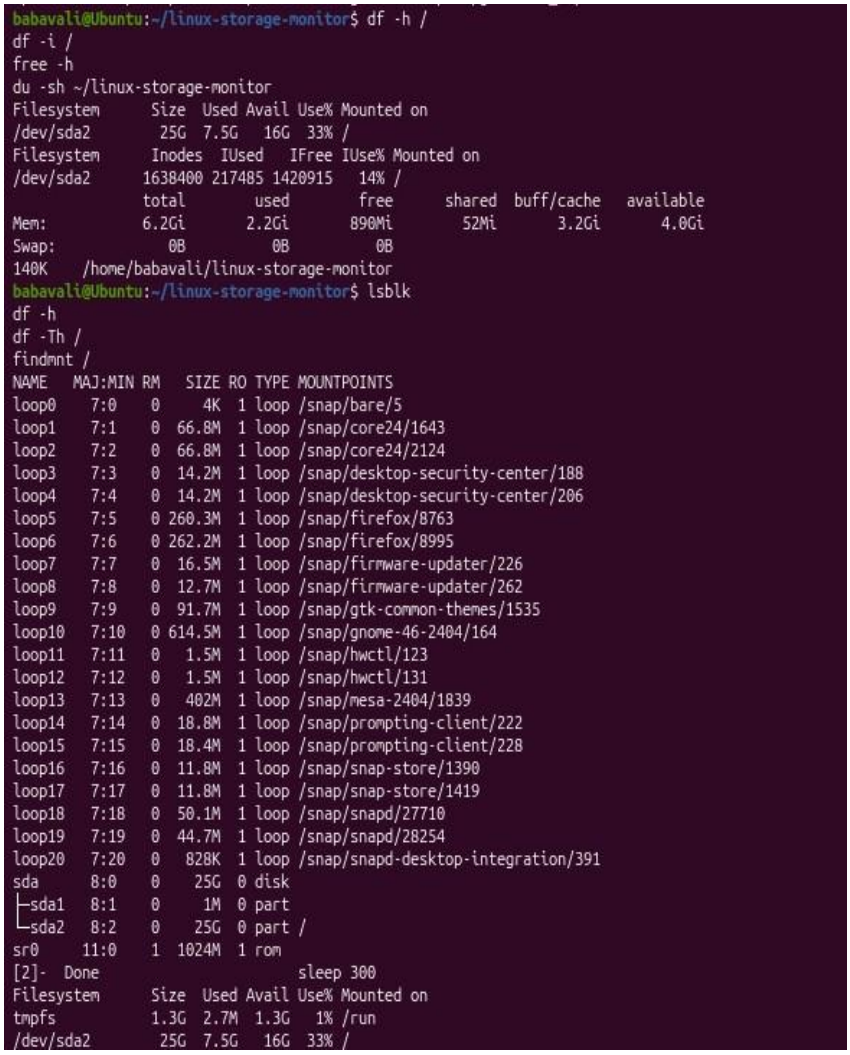


Figure 3.23**:** Disk space monitoring

These commands display block devices, disk space usage, filesystem types, and the filesystem mounted at the root directory

3.6.2 File System Creation

A file system organizes data on a storage device so that files and directories can be stored and accessed. This project uses the existing Linux filesystem for monitoring. No new filesystem creation is required for the monitoring scripts.

The figure 3.24 documents disk space monitoring and supports the explanation in this subsection.

```
</> Bash

df -Th /
lsblk -f
```

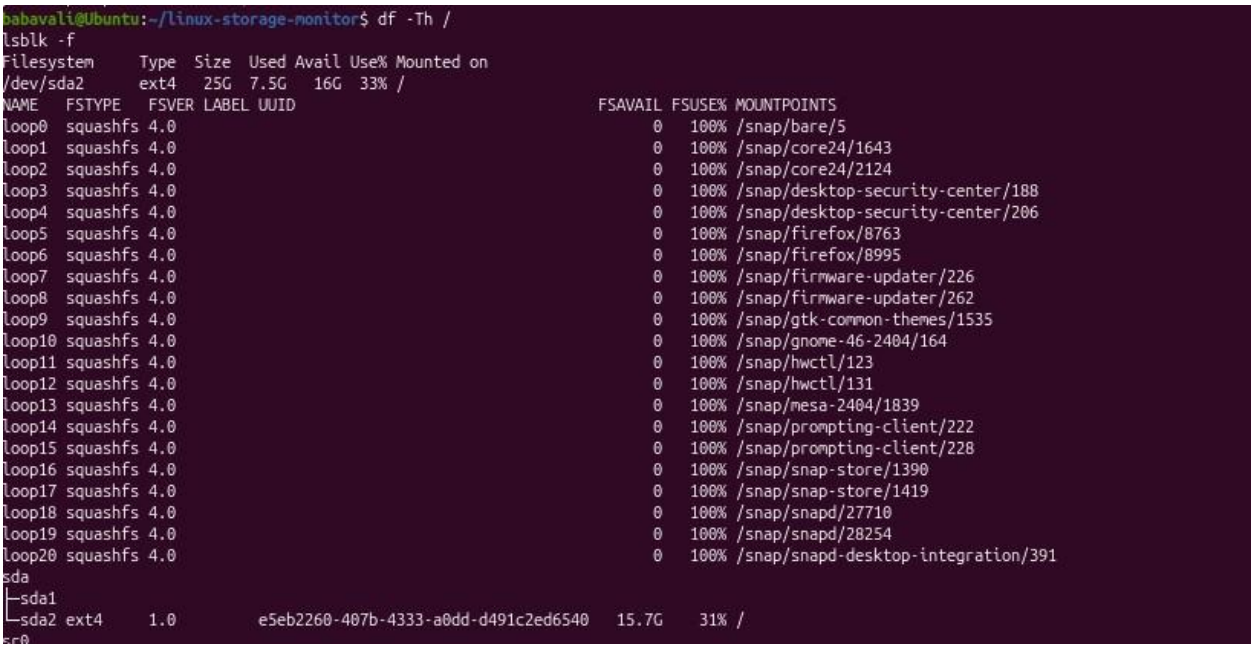


Figure 3.24 : Disk space monitoring

3.6.3 Mounting and Unmounting

Mounting makes a filesystem accessible at a directory called a mount point. The findmnt command displays mounted filesystems. This project monitors the root filesystem and does not require a separate filesystem to be mounted or unmounted.

The figure 3.25 below illustrates disk space monitoring as observed during project setup and validation.

```bash
findmnt
findmnt /
mount | head -n 15
```

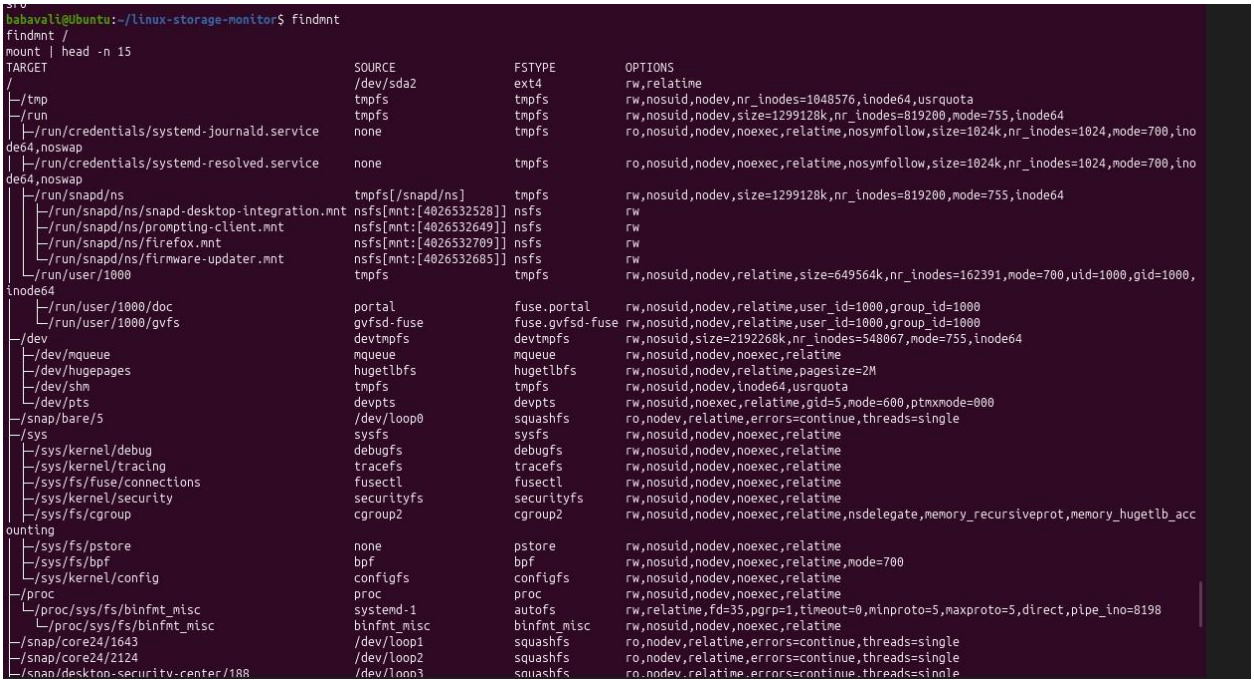


Figure 3.25 : Disk space monitoring

3.6.4 Logical Volume Management (LVM)

Logical Volume Management (LVM) allows administrators to create and manage logical storage volumes. LVM configuration is not part of the current project, so no logical volumes were created or modified. The following commands can be used to check whether LVM devices are present.

The figure 3.26 records the project directory structure and installation and helps verify the corresponding implementation step.

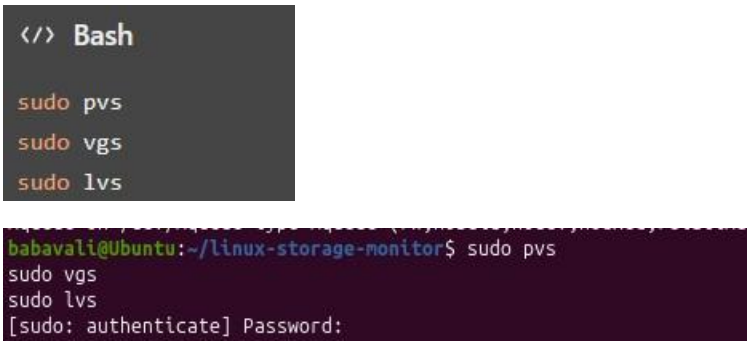


Figure 3.26: Project directory structure and installation

3.6.5 Storage Allocation

Storage allocation refers to the disk space used by files, directories, and applications. The project files are stored in the linux-storage-monitor directory under the user's home directory. The du command is used to determine the storage space occupied by the project, while df shows overall filesystem capacity and available space.

The figure 3.27 documents project directory structure and installation and supports the explanation in this subsection.

```
Bash

du -sh ~/linux-storage-monitor
du -h --max-depth=1 ~/linux-storage-monitor
df -h /
```

```
babavali@Ubuntu:~/linux-storage-monitor$ du -sh ~/linux-storage-monitor
du -h --max-depth=1 ~/linux-storage-monitor
df -h /
140K    /home/babavali/linux-storage-monitor
8.0K    /home/babavali/linux-storage-monitor/config
28K     /home/babavali/linux-storage-monitor/bin
4.0K    /home/babavali/linux-storage-monitor/docs
8.0K    /home/babavali/linux-storage-monitor/backup
16K     /home/babavali/linux-storage-monitor/logs
8.0K    /home/babavali/linux-storage-monitor/reports
48K     /home/babavali/linux-storage-monitor/tests
12K     /home/babavali/linux-storage-monitor/data
140K    /home/babavali/linux-storage-monitor
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda2        25G  7.5G   16G  33% /
```

Figure 3.27**:** Project directory structure and installation

These commands show the total project size, the sizes of its main subdirectories, and the available storage on the root filesystem.

3.6.6 Disk Usage Monitoring

The monitoring system uses the df command and project scripts to check disk usage. It also monitors inode usage, which indicates how many filesystem entries are in use. The warning and critical thresholds are configured in config/monitor.conf.

The current configuration includes:

- MONITOR_PATH=/

- DISK_WARNING=80
- DISK_CRITICAL=90
- INODE_WARNING=80
- INODE_CRITICAL=90

These settings define the monitored path and the warning and critical usage thresholds as percentages.

The figure 3.28 illustrates disk space monitoring as observed during project setup and validation.

```
Bash

cd ~/linux-storage-monitor
cat config/monitor.conf
df -h /
df -i /
bash bin/disk_monitor.sh
bash bin/inode_monitor.sh
```

```
babavali@Ubuntu:~/linux-storage-monitor$ cd ~/linux-storage-monitor
cat config/monitor.conf
df -h /
df -i /
bash bin/disk_monitor.sh
bash bin/inode_monitor.sh
# Disk utilization thresholds
DISK_WARNING=80
DISK_CRITICAL=90

# Inode utilization thresholds
INODE_WARNING=80
INODE_CRITICAL=90

# Filesystem to monitor
MONITOR_PATH=/
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda2        25G  7.5G   16G  33% /
Filesystem      Inodes  IUsed   IFree IUse% Mounted on
/dev/sda2      1638400 217539 1420861   14% /
===== Disk Utilization Monitor =====
Date       : 2026-10-12 13:21:15
Filesystem : /dev/sda2
Mountpoint : /
Total      : 25623780
Used       : 7843640
Available  : 16453192
Usage      : 33%
Status     : NORMAL

Monitoring result saved to logs/storage_monitor.log
===== Inode Utilization Monitor =====
Date            : 2026-10-12 13:21:15
Filesystem      : /dev/sda2
Mountpoint      : /
Total Inodes    : 1638400
Used Inodes     : 217539
Free Inodes     : 1420861
Inode Usage     : 14%
Status          : NORMAL

Monitoring result saved to logs/inode_monitor.log
```

Figure 3.28**:** Disk space monitoring

The configuration file can be checked to verify the thresholds. The disk and inode monitoring scripts display the current storage conditions according to their implementation.

3.6.7 Storage Architecture and Configuration Details

The storage architecture of the project consists of the existing Linux filesystem, the project directory, configuration files, monitoring scripts, log files, storage history, and generated reports.

The system monitors the existing root filesystem rather than creating a new partition or logical volume. Monitoring scripts read storage information, compare usage against configured thresholds, and store results in logs and history files. Reports and backup archives are maintained within the project directory.

This figure 3.29 presents the project directory structure and installation, providing visual evidence of the configuration or result described above.

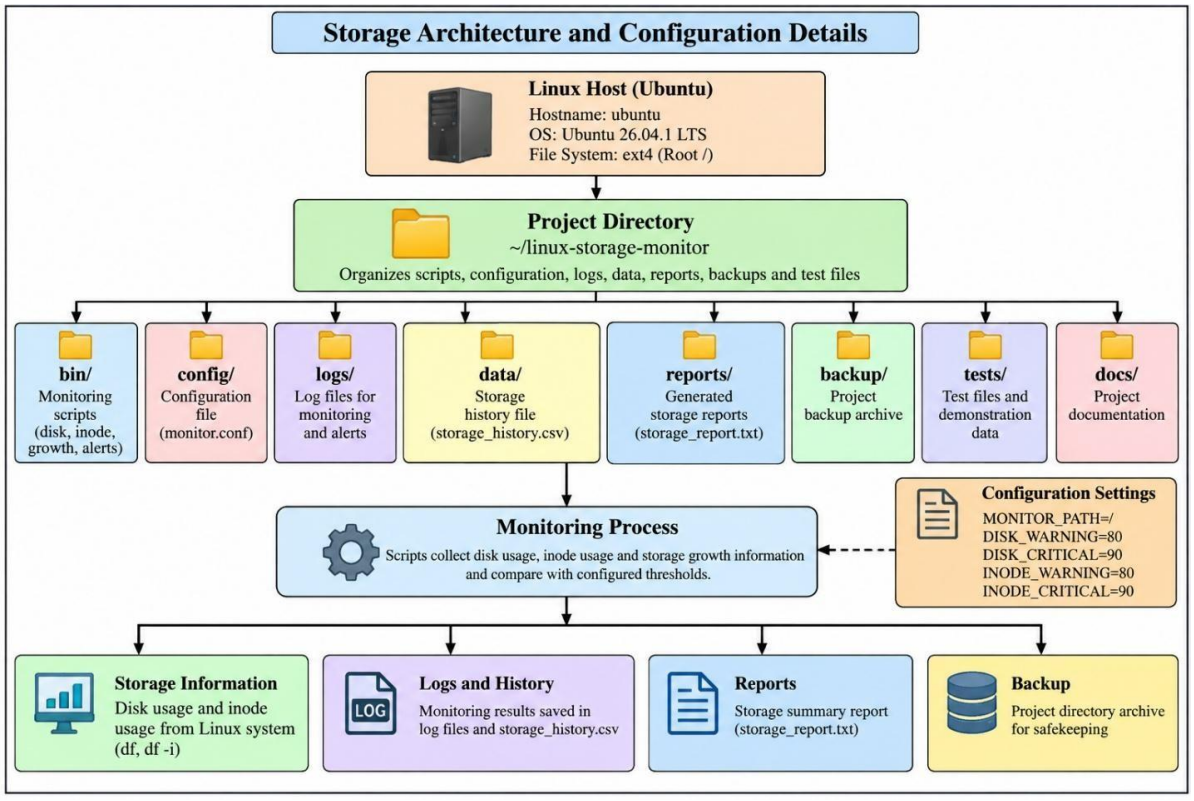


Figure 3.29 : Project directory structure and installation

3.7 Backup and Automation

Backup and automation are important features of the Linux Storage Monitoring System. Backup helps protect project files from accidental loss, while automation allows monitoring scripts and report generation tasks to run at scheduled intervals without requiring manual execution each time.

3.7.1 Backup Strategy

The project uses a compressed archive to create a backup of important project files. The backup is stored as project_backup.tar.gz. The tar utility packages the selected files and directories, while gzip compression reduces the archive size. This backup helps preserve project scripts, configuration files, tests, and other important project data.

3.7.2 Backup Location

The backup archive is stored in the project's backup directory.

Backup path: ~/linux-storage-monitor/backup/project_backup.tar.gz

The archive can be checked using the following commands:

```bash
cd ~/linux-storage-monitor
ls -lh backup/project_backup.tar.gz
tar -tzf backup/project_backup.tar.gz
```

```
babavali@Ubuntu:~/linux-storage-monitor$ cd ~/linux-storage-monitor
ls -lh backup/project_backup.tar.gz
tar -tzf backup/project_backup.tar.gz
-rw-rw-r-- 1 babavali babavali 1.8K Oct 10 19:53 backup/project_backup.tar.gz
README.md
config/
config/monitor.conf
tests/
tests/admin_demo/
tests/admin_demo/group_test.txt
tests/test_alerts.sh
```

Figure 3.30: Project directory structure and installation

The first command displays the archive's size and details, while the second lists its contents without extracting the files.

The figure 3.30 captures the relevant output for project directory structure and installation, showing how this part of the Linux project was checked

3.7.3 Backup Frequency

The project contains a backup archive. A recurring backup schedule should not be assumed unless a backup job has been configured. The monitoring scripts and report generation tasks are scheduled separately using Cron.

The backup archive can be checked periodically to confirm that it exists and contains the expected files.

3.7.4 Shell Script Automation

Bash shell scripts are used to automate the monitoring tasks. The project includes scripts for disk usage monitoring, inode usage monitoring, storage growth tracking, alert monitoring, and report generation.

The scripts are stored in the bin directory:

- disk_monitor.sh – monitors disk space usage.
- inode_monitor.sh – monitors inode utilization.
- storage_growth.sh – tracks storage growth.
- alert_monitor.sh – handles storage alert monitoring.
- generate_report.sh – generates the storage summary report.

These scripts reduce repetitive manual work by collecting storage information and recording results automatically.

3.7.5 Cron Configuration

Cron is used to schedule the monitoring scripts and report generation. The Cron service runs scheduled tasks at specified intervals.

The project's scheduled tasks include monitoring scripts running every five minutes and report generation every fifteen minutes.

Commands to verify the service and scheduled jobs:

```bash
systemctl is-active cron
systemctl status cron --no-pager
crontab -l
```

```
babavali@Ubuntu:~/linux-storage-monitor$ systemctl is-active cron
systemctl status cron --no-pager
crontab -l
active
● cron.service - Regular background program processing daemon
     Loaded: loaded (/usr/lib/systemd/system/cron.service; enabled; preset: enabled)
     Active: active (running) since Mon 2026-10-12 11:29:20 UTC; 1h 58min ago
 Invocation: cf6d9a6862f846c98f904a5ad4658d3a
       Docs: man:cron(8)
   Main PID: 1056 (cron)
      Tasks: 1 (limit: 5138)
     Memory: 872K (peak: 18M)
        CPU: 1min 5.230s
     CGroup: /system.slice/cron.service
             └─1056 /usr/sbin/cron -f -P

Oct 12 13:25:01 Ubuntu CRON[13590]: (babavali) CMD (/home/babavali/linux-storage-monitor/bin/inode_monitor.sh)
Oct 12 13:25:01 Ubuntu CRON[13592]: (babavali) CMD (/home/babavali/linux-storage-monitor/bin/alert_monitor.sh)
Oct 12 13:25:02 Ubuntu CRON[13584]: (CRON) info (No MTA installed, discarding output)
Oct 12 13:25:02 Ubuntu CRON[13584]: pam_unix(cron:session): session closed for user babavali
Oct 12 13:25:03 Ubuntu CRON[13583]: (CRON) info (No MTA installed, discarding output)
Oct 12 13:25:03 Ubuntu CRON[13583]: pam_unix(cron:session): session closed for user babavali
Oct 12 13:25:03 Ubuntu CRON[13582]: (CRON) info (No MTA installed, discarding output)
Oct 12 13:25:03 Ubuntu CRON[13582]: pam_unix(cron:session): session closed for user babavali
Oct 12 13:25:03 Ubuntu CRON[13581]: (CRON) info (No MTA installed, discarding output)
Oct 12 13:25:03 Ubuntu CRON[13581]: pam_unix(cron:session): session closed for user babavali
*/5 * * * * /home/babavali/linux-storage-monitor/bin/disk_monitor.sh
*/5 * * * * /home/babavali/linux-storage-monitor/bin/inode_monitor.sh
*/5 * * * * /home/babavali/linux-storage-monitor/bin/storage_growth.sh
*/5 * * * * /home/babavali/linux-storage-monitor/bin/alert_monitor.sh
*/15 * * * * /home/babavali/linux-storage-monitor/bin/generate_report.sh
babavali@Ubuntu:~/linux-storage-monitor$
```

Figure 3.31: Cron service and scheduled automation

The crontab -l command displays the current user's scheduled jobs. The Cron service status confirms whether the scheduling service is active.

3.7.6 Log Generation

The monitoring system records information in log files so that monitoring activity and alerts can be reviewed later. The project includes the following log files:

- logs/storage_monitor.log – stores storage monitoring information.
- logs/administrator_alerts.log – stores administrator alert information.
- data/storage_history.csv – stores storage history data.
- reports/storage_report.txt – contains the generated storage report.

Commands to inspect the logs and report:

```bash
cd ~/linux-storage-monitor
tail -n 20 logs/storage_monitor.log
tail -n 20 logs/administrator_alerts.log
cat reports/storage_report.txt
```

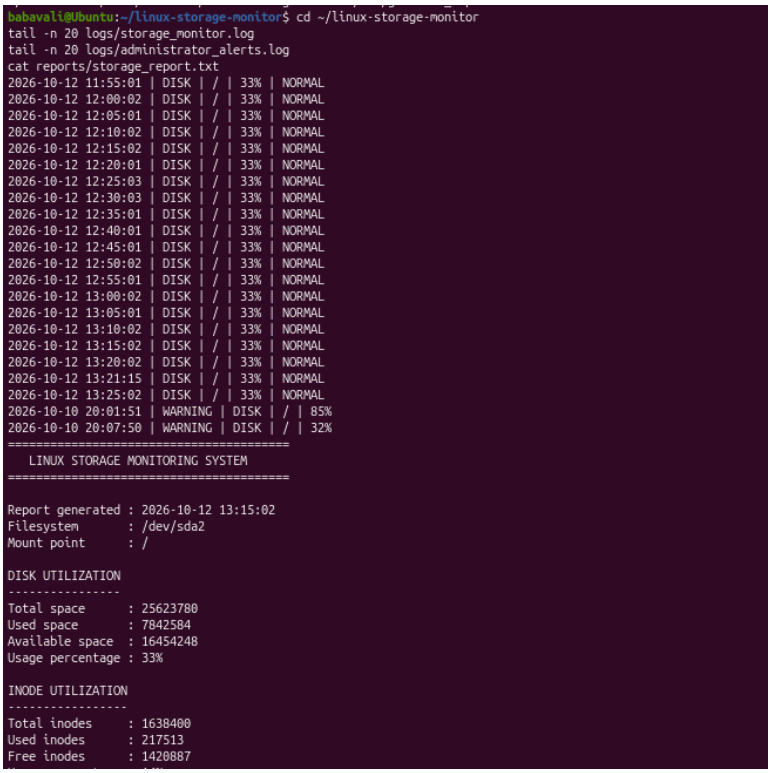


Figure 3.32 : Storage threshold and alert checks

This figure 3.32 presents the storage threshold and alert checks, providing visual evidence of the configuration or result described above.

These commands help verify monitoring activity, review alerts, and inspect the generated report.

3.7.7 Recovery Procedure

Recovery involves extracting selected files from the backup archive and checking that the recovered files match the originals. The project backup can be inspected before extraction to identify the required files.

To list the archive contents:

```bash
cd ~/linux-storage-monitor
tar -tzf backup/project_backup.tar.gz
```

```
babavali@Ubuntu:~/linux-storage-monitor$ cd ~/linux-storage-monitor
tar -tzf backup/project_backup.tar.gz
README.md
config/
config/monitor.conf
tests/
tests/admin_demo/
tests/admin_demo/group_test.txt
tests/test_alerts.sh
```

Figure 3.33 : Storage threshold and alert checks

To extract the archive into a separate recovery directory:

```bash
mkdir -p /tmp/storage-monitor-recovery
tar -xzf backup/project_backup.tar.gz -C /tmp/storage-monitor-recovery
```

```
babavali@Ubuntu:~/linux-storage-monitor$ mkdir -p /tmp/storage-monitor-recovery
tar -xzf backup/project_backup.tar.gz -C /tmp/storage-monitor-recovery
```

Figure 3.34: Storage threshold and alert checks

The figure 3.34 documents storage threshold and alert checks and supports the explanation in this subsection.

After extraction, the recovered files can be inspected and compared with the original files. For example, use diff to compare matching text files. An empty output indicates that the compared files have no differences.

Recovery testing should be performed in a separate directory to avoid overwriting the original project files.

3.7.8 Benefits of Automation

Automation reduces manual administrative effort by running storage checks and report generation at scheduled intervals. It helps maintain regular monitoring, records storage information for later review, and makes it easier to identify conditions that may require administrator attention.

Using Cron and Bash scripts improves consistency and reduces the need to execute each monitoring command manually. The backup archive also provides a way to preserve and recover selected project files when needed.

Result

The project demonstrates compressed backup storage, scheduled monitoring through Cron, automatic report generation, log recording, and recovery testing of selected files. These features support more consistent storage administration and reduce repetitive manual tasks.

3.8 Security and Access Control

Security and access control are important parts of the Linux Storage Monitoring System. They help protect project files, control user access, and reduce the risk of unauthorized modifications. The project demonstrates Linux user and group management, file permissions, ownership, password management, and service status verification.

3.8.1 User Permissions

Linux user and group permissions help control access to files and directories. In this project, the storagetest user and storageproject group are used to demonstrate access control. The user is added to the project group so that group permissions can be tested.

```bash
id storagetest
getent group storageproject
groups storagetest
```

```
babavali@Ubuntu:~/linux-storage-monitor$ id storagetest
getent group storageproject
groups storagetest
uid=1009(storagetest) gid=1009(storagetest) groups=1009(storagetest),100(users),1008(storageproject)
storageproject:x:1008:storagetest
storagetest : storagetest users storageproject
```

Figure 3.35: Test user account configuration

3.8.2 File Permissions

Linux file permissions determine whether users can read, write, or execute files. The project demonstrates directory and file permissions using a shared directory owned by root and the storageproject group. The directory uses permission mode 2770, and the demonstration file uses permission mode 640.

The figure 3.36 captures the relevant output for test user account configuration, showing how this part of the Linux project was checked.

Commands:

```
Bash

ls -ld /tmp/storageproject_demo
ls -l /tmp/storageproject_demo
stat -c '%A %a %n' /tmp/storageproject_demo/group_test.txt
```

```
babavali@Ubuntu:~/linux-storage-monitor$ ls -ld /tmp/storageproject_demo
ls -l /tmp/storageproject_demo
stat -c '%A %a %n' /tmp/storageproject_demo/group_test.txt
drwxrws--- 2 root storageproject 60 Oct 12 11:54 /tmp/storageproject_demo
```

Figure 3.36 : Test user account configuration

The directory's 2770 permission mode enables owner and group access and applies the set-group-ID bit. The file's 640 permission mode gives the owner read and write access, the group read access, and others no access.

3.8.3 Firewall Configuration

A firewall helps control incoming and outgoing network traffic. The Ubuntu firewall status was checked using the UFW utility. The observed status was inactive, so an enabled firewall should not be claimed as an implemented security measure.

Command:

sudo ufw status verbose

This command displays the firewall status and any configured rules. Enabling a firewall should be done only after reviewing the system's network requirements.

3.8.4 SSH Configuration

Secure Shell (SSH) provides encrypted remote access to a Linux system. During the project checks, the SSH service was not found. Therefore, SSH-based remote access configuration was not implemented or verified as part of this project.

Command:

```bash
systemctl status ssh --no-pager
```

The command checks the SSH service status.

3.8.5 Service Access Control

Linux services can be inspected and managed using systemctl. The Cron service is used in this project to schedule monitoring scripts and report generation. Checking its status helps verify that the scheduling service is operating.

Commands:

```bash
systemctl is-active cron
systemctl status cron --no-pager
```

Service status checks help administrators identify whether a required service is active and investigate service-related issues.

```
babavali@Ubuntu:~/linux-storage-monitor$ systemctl is-active cron
systemctl status cron --no-pager
active
● cron.service - Regular background program processing daemon
     Loaded: loaded (/usr/lib/systemd/system/cron.service; enabled; preset: enabled)
     Active: active (running) since Mon 2026-10-12 11:29:20 UTC; 2h 12min ago
 Invocation: cf6d9a6862f846c98f904a5ad4658d3a
       Docs: man:cron(8)
   Main PID: 1056 (cron)
      Tasks: 1 (limit: 5138)
     Memory: 872K (peak: 18M)
        CPU: 1min 21.865s
     CGroup: /system.slice/cron.service
             └─1056 /usr/sbin/cron -f -P

Oct 12 13:40:01 Ubuntu CRON[14245]: (babavali) CMD (/home/babavali/linux-storage-monitor/bin/disk_monitor.sh)
Oct 12 13:40:01 Ubuntu CRON[14248]: (babavali) CMD (/home/babavali/linux-storage-monitor/bin/storage_growth.sh)
Oct 12 13:40:01 Ubuntu CRON[14237]: (CRON) info (No MTA installed, discarding output)
Oct 12 13:40:01 Ubuntu CRON[14237]: pam_unix(cron:session): session closed for user babavali
Oct 12 13:40:01 Ubuntu CRON[14235]: (CRON) info (No MTA installed, discarding output)
Oct 12 13:40:01 Ubuntu CRON[14235]: pam_unix(cron:session): session closed for user babavali
Oct 12 13:40:01 Ubuntu CRON[14238]: (CRON) info (No MTA installed, discarding output)
Oct 12 13:40:01 Ubuntu CRON[14238]: pam_unix(cron:session): session closed for user babavali
Oct 12 13:40:01 Ubuntu CRON[14236]: (CRON) info (No MTA installed, discarding output)
Oct 12 13:40:01 Ubuntu CRON[14236]: pam_unix(cron:session): session closed for user babavali
```

Figure 3.37 : Test user account configuration

3.8.6 Password Policies

Linux user accounts can be protected using passwords. The project includes user and password management demonstrations for the test account. Passwords should be kept confidential and should follow the organization's password requirements.

Commands:

```bash
sudo passwd storagetest
sudo passwd -S storagetest
```

The first command allows an administrator to set or change the test user's password. The second displays password status information.

3.8.7 Security Monitoring

Security-related checks include reviewing file permissions, checking user and group membership, inspecting service status, and examining monitoring logs. These checks help identify access configuration issues and provide information for troubleshooting.

Commands:

```bash
cd ~/linux-storage-monitor
ls -l bin/
tail -n 20 logs/storage_monitor.log
tail -n 20 logs/administrator_alerts.log
```

These commands display script permissions and recent monitoring and alert log entries:

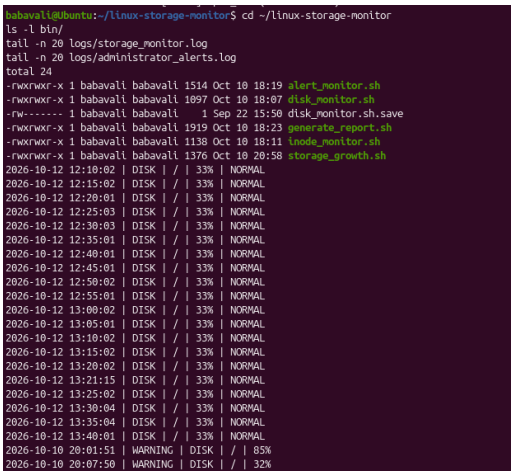


Figure 3.38: Test user account configuration

3.8.8 Security Benefits

The implemented measures improve system security in the following ways:

- User and group management: helps organize access to shared project files.
- File permissions: limits reading, writing, and execution according to ownership and permission settings.
- Password management: supports controlled access to user accounts.
- Service monitoring: helps administrators verify the status of scheduled services.
- Log inspection: supports troubleshooting and review of monitoring activity.
- Firewall and SSH checks: help identify the current state of network security components.

4.TEST AND VALIDATION

4.1 Implementation Results

The Linux Storage Monitoring System was implemented using Bash shell scripts, Linux administration commands, configuration files, Cron scheduling, log files, and backup utilities. The system monitors disk space and inode usage, records storage information, generates reports, and supports backup and recovery testing.

The implementation results demonstrate the use of Linux system administration concepts, including file management, user and group permissions, package management, process monitoring, service management, storage monitoring, and task automation.

4.1.1 Screenshots and Command Outputs

Screenshots of the terminal outputs provide evidence of the implementation. The project directory structure, monitoring script outputs, configuration settings, log files, generated reports, and backup archive can be included to demonstrate the operation of the system.

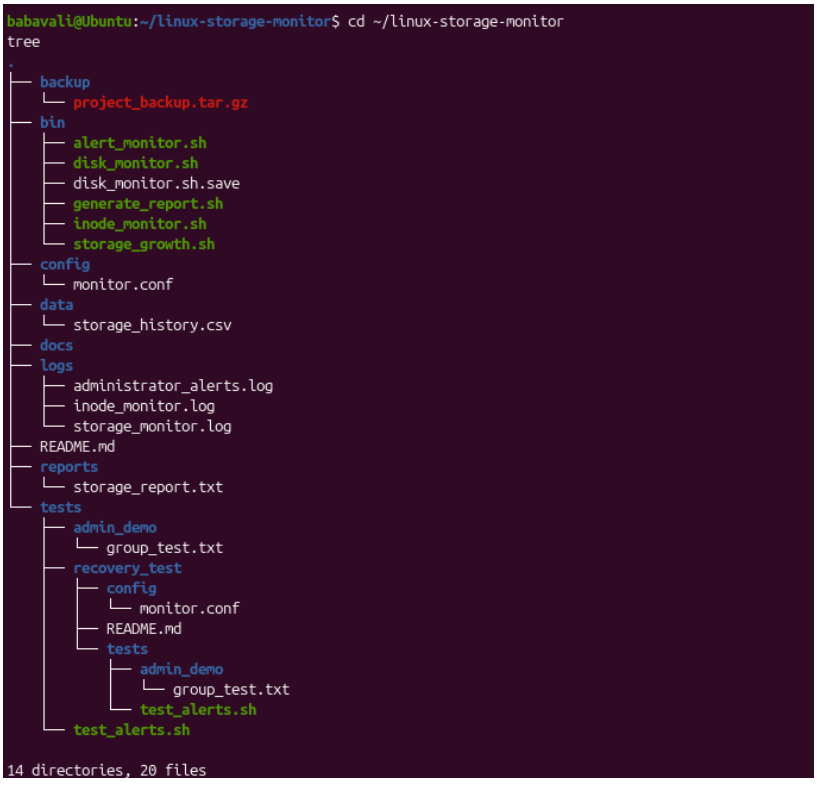


Figure 4.1 **:** Project directory structure and installation

4.1.2 Storage Monitoring Results

The system monitors disk space and inode usage to identify storage conditions that may require attention.

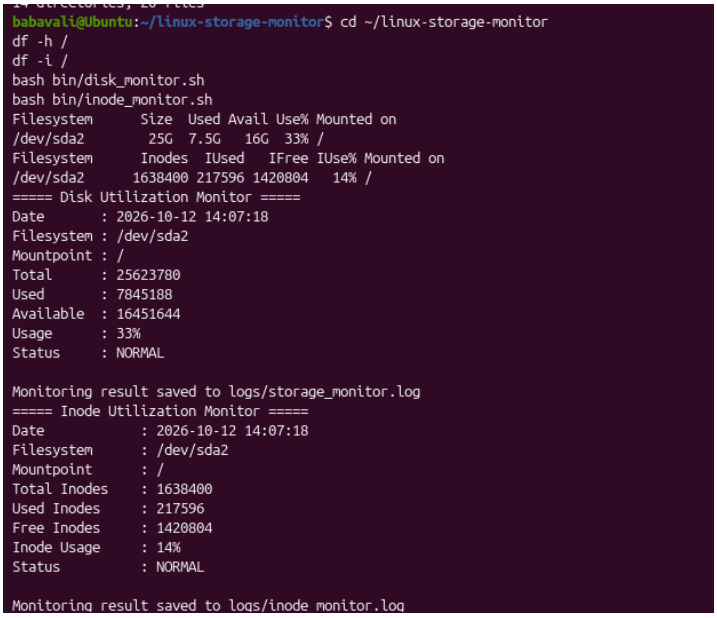


Figure 4.2 : Project directory structure and installation

The observed disk usage was approximately 33%, while inode usage was approximately 14%. Both values were below the configured warning threshold of 80% at the time of observation.

4.1.3 Configuration Results

The monitoring configuration defines the filesystem path and the warning and critical thresholds used by the scripts.

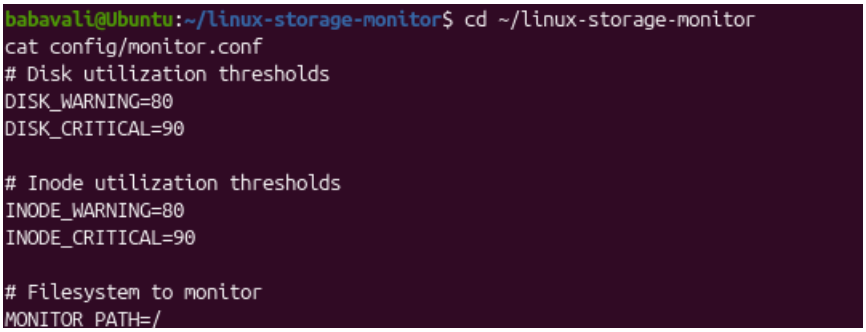


Figure 4.3: Project directory structure and installation

The configuration includes MONITOR_PATH=/, disk warning and critical thresholds of 80% and 90%, and inode warning and critical thresholds of 80% and 90%.

4.1.4 System Status and Automation Results

Cron is used to execute the monitoring scripts and report generation tasks at scheduled intervals.

```
babavali@Ubuntu:~/linux-storage-monitor$ systemctl is-active cron
systemctl status cron --no-pager
crontab -l
active
● cron.service - Regular background program processing daemon
     Loaded: loaded (/usr/lib/systemd/system/cron.service; enabled; preset: enabled)
     Active: active (running) since Mon 2026-10-12 11:29:20 UTC; 2h 42min ago
 Invocation: cf6d9a6862f846c98f904a5ad4658d3a
       Docs: man:cron(8)
   Main PID: 1056 (cron)
      Tasks: 1 (limit: 5138)
     Memory: 884K (peak: 18M)
        CPU: 2min 11.224s
     CGroup: /system.slice/cron.service
             └─1056 /usr/sbin/cron -f -P

Oct 12 14:10:03 Ubuntu CRON[15327]: (babavali) CMD (/home/babavali/linux-storage-monitor/bin/disk_monitor.sh)
Oct 12 14:10:03 Ubuntu CRON[15328]: (babavali) CMD (/home/babavali/linux-storage-monitor/bin/storage_growth.sh)
Oct 12 14:10:04 Ubuntu CRON[15316]: (CRON) info (No MTA installed, discarding output)
Oct 12 14:10:04 Ubuntu CRON[15316]: pam_unix(cron:session): session closed for user babavali
Oct 12 14:10:04 Ubuntu CRON[15317]: (CRON) info (No MTA installed, discarding output)
Oct 12 14:10:04 Ubuntu CRON[15314]: (CRON) info (No MTA installed, discarding output)
Oct 12 14:10:04 Ubuntu CRON[15317]: pam_unix(cron:session): session closed for user babavali
Oct 12 14:10:04 Ubuntu CRON[15314]: pam_unix(cron:session): session closed for user babavali
Oct 12 14:10:04 Ubuntu CRON[15315]: (CRON) info (No MTA installed, discarding output)
Oct 12 14:10:04 Ubuntu CRON[15315]: pam_unix(cron:session): session closed for user babavali
*/5 * * * * /home/babavali/linux-storage-monitor/bin/disk_monitor.sh
*/5 * * * * /home/babavali/linux-storage-monitor/bin/inode_monitor.sh
*/5 * * * * /home/babavali/linux-storage-monitor/bin/storage_growth.sh
*/5 * * * * /home/babavali/linux-storage-monitor/bin/alert_monitor.sh
*/15 * * * * /home/babavali/linux-storage-monitor/bin/generate_report.sh
```

Figure 4.4 **:** Project directory structure and installation

The Cron service was verified as active. The configured schedule runs monitoring scripts every five minutes and generates reports every fifteen minutes. The scheduled configuration reduces the need for repeated manual execution.

4.1.5 Backup Results

A compressed backup archive is stored in the project's backup directory.

```
babavali@Ubuntu:~/linux-storage-monitor$ cd ~/linux-storage-monitor
ls -lh backup/project_backup.tar.gz
tar -tzf backup/project_backup.tar.gz
-rw-rw-r-- 1 babavali babavali 1.8K Oct 10 19:53 backup/project_backup.tar.gz
README.md
config/
config/monitor.conf
tests/
tests/admin_demo/
tests/admin_demo/group_test.txt
tests/test_alerts.sh
```

Figure 4.5 : Project directory structure and installation

The backup archive was verified, and its contents were listed successfully. This confirms that the archive exists and can be inspected.

4.1.6 Log and Report Results

The system stores monitoring activity, administrator alerts, storage history, and generated reports in separate files.

```
babavali@Ubuntu:~/linux-storage-monitor$ cd ~/linux-storage-monitor
tail -n 15 logs/storage_monitor.log
tail -n 15 logs/administrator_alerts.log
cat reports/storage_report.txt
2026-10-12 13:10:02 | DISK | / | 33% | NORMAL
2026-10-12 13:15:02 | DISK | / | 33% | NORMAL
2026-10-12 13:20:02 | DISK | / | 33% | NORMAL
2026-10-12 13:21:15 | DISK | / | 33% | NORMAL
2026-10-12 13:25:02 | DISK | / | 33% | NORMAL
2026-10-12 13:30:04 | DISK | / | 33% | NORMAL
2026-10-12 13:35:04 | DISK | / | 33% | NORMAL
2026-10-12 13:40:01 | DISK | / | 33% | NORMAL
2026-10-12 13:45:02 | DISK | / | 33% | NORMAL
2026-10-12 13:50:03 | DISK | / | 33% | NORMAL
2026-10-12 13:55:03 | DISK | / | 33% | NORMAL
2026-10-12 14:00:04 | DISK | / | 33% | NORMAL
2026-10-12 14:05:04 | DISK | / | 33% | NORMAL
2026-10-12 14:07:18 | DISK | / | 33% | NORMAL
2026-10-12 14:10:04 | DISK | / | 33% | NORMAL
2026-10-10 20:01:51 | WARNING | DISK | / | 85%
2026-10-10 20:07:50 | WARNING | DISK | / | 32%
========================================
   LINUX STORAGE MONITORING SYSTEM
========================================

Report generated : 2026-10-12 14:00:04
Filesystem       : /dev/sda2
Mount point      : /

DISK UTILIZATION
----------------
Total space      : 25623780
Used space       : 7845908
Available space  : 16450924
Usage percentage : 33%
```

Figure 4.6 : : Project directory structure and installation

The log files help review monitoring activity and alerts, while the report provides a summary of storage monitoring results.

4.1.7 Security Configuration

The project demonstrated user and group management, file ownership, permission configuration, and access testing. The test user successfully accessed the demonstration file through group permissions. The UFW firewall was found to be inactive, and the SSH service was not found during the checks.

```
babavali@Ubuntu:~/linux-storage-monitor$ id storagetest
getent group storageproject
ls -ld /tmp/storageproject_demo
ls -l /tmp/storageproject_demo
sudo -u storagetest cat /tmp/storageproject_demo/group_test.txt
uid=1009(storagetest) gid=1009(storagetest) groups=1009(storagetest),100(users),1008(storageproject)
storageproject:x:1008:storagetest
drwxrws--- 2 root storageproject 60 Oct 12 11:54 /tmp/storageproject_demo
ls: cannot open directory '/tmp/storageproject_demo': Permission denied
[sudo: authenticate] Password:
Linux storage permission demonstration
```

Figure 4.7 **:** Project directory structure and installation

Overall, the implementation demonstrates a basic Linux storage monitoring solution that combines storage checks, logging, report generation, scheduled execution, and backup validation.

4.2 Testing and Validation

Testing and validation were performed to verify the main functions of the Linux Storage Monitoring System. The test cases cover user and group management, permissions, package installation, service status, backup creation, recovery testing, automation, and access control.

Test Case	Expected Result	Actual Result	Status
User creation and group verification	User and group exist, and membership is configured correctly.	storagetest and storageproject were verified.	Pass
Permission configuration	Correct access is granted to authorized users.	The test user successfully accessed the shared demonstration file.	Pass
Package installation	Required package is installed successfully.	tree was installed and its version was checked.	Pass
Service configuration	Cron service runs successfully.	Cron service status was verified as active.	Pass
Backup execution	Backup archive exists and contains project files.	Archive was present and its contents were listed.	Pass
Data recovery	Selected files are recovered and match originals.	Selected recovered files were compared with originals using diff.	Pass
Automation	Scheduled tasks are configured correctly.	Cron jobs were configured for monitoring and report generation.	Configuration verified

Storage monitoring	Disk and inode usage are checked against thresholds.	Usage was below the warning thresholds during observation.	Pass
Security test	Access follows the configured file permissions.	Group-based access to the demonstration file was successful.	Pass for demonstrated test

TABLE 1: TESTING AND VALIDATION

4.2.2 Recovery Testing

The backup recovery test checks whether selected files can be extracted into a separate directory and compared with their original versions.

4.2.3 Package Verification

The package test verifies that the tree utility is installed and available.

4.2.4 Testing Summary

The tests demonstrate the main features of the Linux Storage Monitoring System, including storage monitoring, configuration verification, user and group permissions, package management, Cron scheduling, backup archive inspection, and selected-file recovery. These results show how Linux administration tools and Bash scripts can support routine storage monitoring and reduce repetitive manual work.

5. Conclusion and Future Scope

5.1 Conclusion

The Linux Storage Monitoring System provided practical experience in Linux system administration by implementing disk space monitoring, inode usage monitoring, storage growth tracking, alert monitoring, and report generation. The project used Bash shell scripts, Linux commands, configuration files, and Cron scheduling to automate routine storage monitoring tasks.

The project also demonstrated user and group management, file and directory permissions, package and process management, storage usage analysis, backup creation, recovery testing, and log management. The implementation was tested by checking monitoring outputs, verifying scheduled tasks, inspecting the backup archive, and comparing selected recovered files with their original versions.

The project achieved its main objective of demonstrating a basic Linux storage monitoring solution. It helped develop practical knowledge of Linux administration, storage management, automation, access control, and system maintenance.

5.2 Future Scope

The project can be enhanced in the following ways:

- Advanced monitoring and alerting: Introduce real-time monitoring and notifications when disk or inode usage reaches warning or critical thresholds.
- Automated backup and recovery: Schedule regular backups and improve recovery procedures to protect important project files.
- Centralized logging: Store monitoring logs in a centralized location to simplify analysis and troubleshooting.
- Improved security: Strengthen file permissions, password policies, firewall configuration, and access control.
- Graphical dashboard: Develop a dashboard to display disk usage, inode utilization, storage growth, and alert history.

- Containerization: Package the monitoring system using container technologies to improve portability and deployment.
- Automated infrastructure management: Use configuration management tools to simplify deployment and maintenance across multiple Linux systems.

References

1. Ubuntu Documentation, Ubuntu Server Documentation. Available at: https://ubuntu.com/server/docs/.

Ubuntu

2. GNU Project, GNU Bash Reference Manual. Available at: https://www.gnu.org/software/bash/manual/bash.html.

GNU

3. GNU Project, GNU Coreutils Manual. Available at: https://www.gnu.org/software/coreutils/manual/coreutils.html.

GNU

4. Ubuntu Manpages, crontab — Maintains Crontab Files for Individual Users. Available at: https://manpages.ubuntu.com/manpages/noble/man1/crontab.1.html.

maintains crontab files for individual users

5. GNU Project, GNU Tar Manual. Available at: https://www.gnu.org/software/tar/manual/tar.html.

GNU

6. Linux Manual Pages Project, Linux Manual Pages (man pages). Available at: https://www.kernel.org/doc/man-pages/.

7. Ubuntu Documentation, Managing Software. Available through the Ubuntu Server Documentation

Appendices

Appendix A: System Architecture

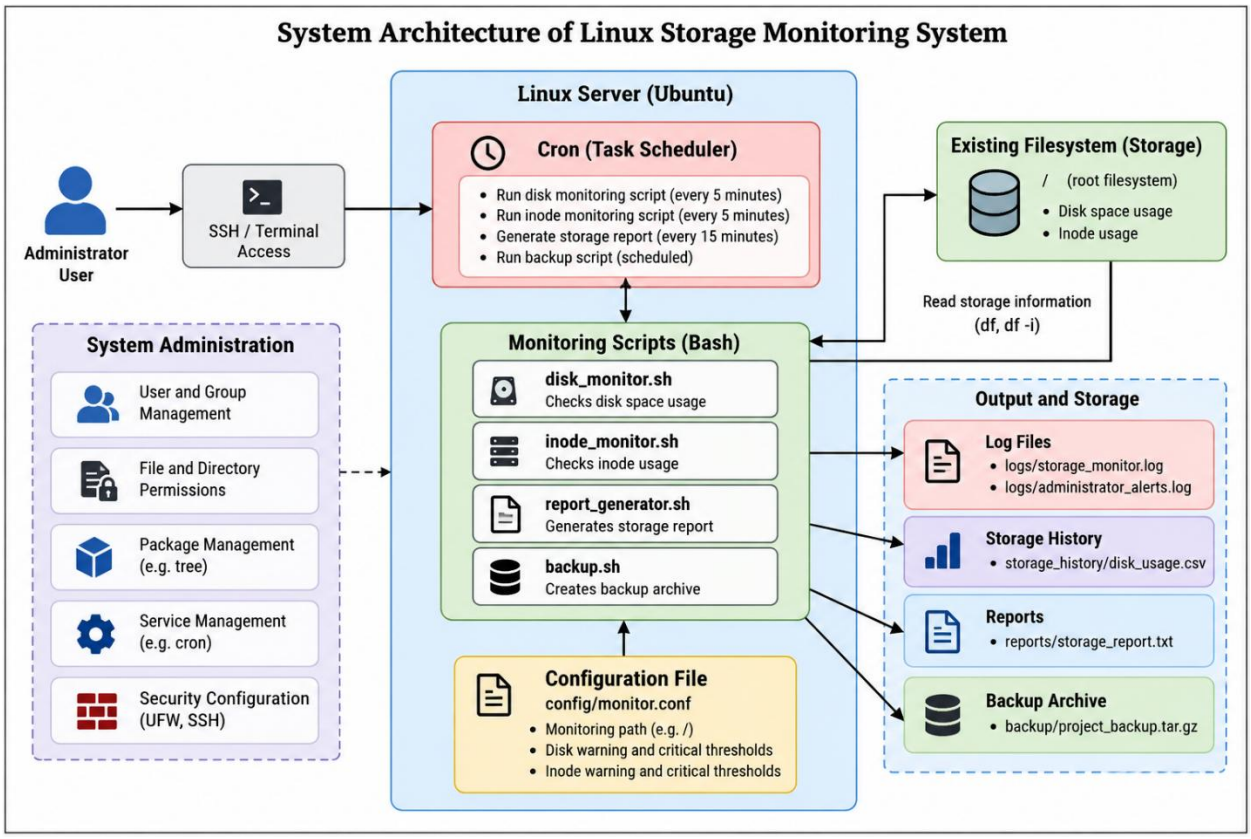


Figure A1 : System Architecture of Linux Storage Monitoring System

Appendix B: Linux Commands and Configuration

This appendix presents important Linux commands, configuration files, and terminal outputs used in the Linux Storage Monitoring System. The screenshots already included in this document provide evidence of the commands and results.

B.1 System Information Commands

These commands display the current user, hostname, and Linux distribution details.

```
whoami
hostname
hostnamectl
lsb_release -a
```

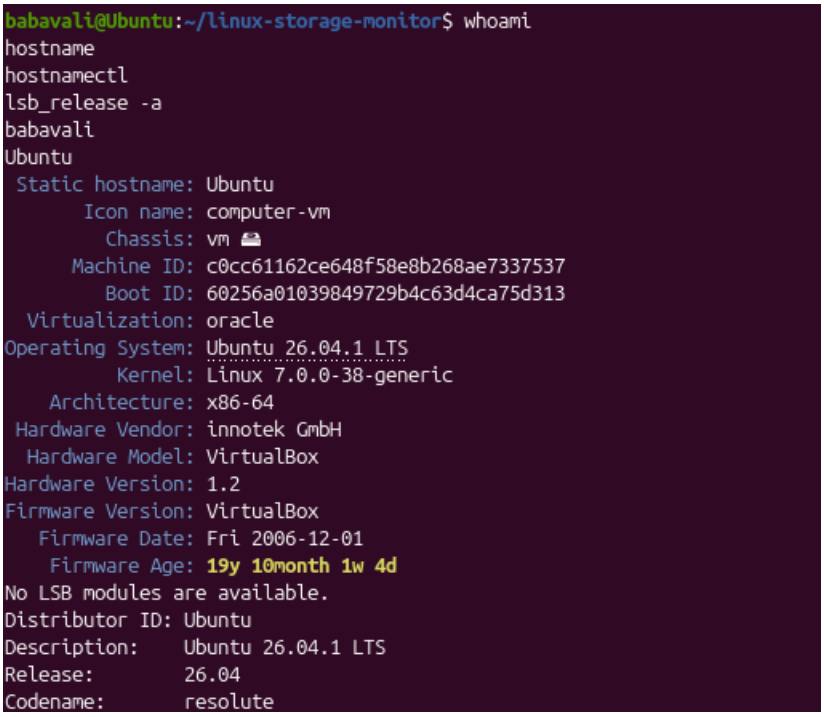


Figure B1 : Ubuntu version and filesystem details

B.2 Project Directory Structure

The project directory organizes scripts, configuration, logs, data, reports, backups, tests, and documentation.

```
cd ~/linux-storage-monitor
pwd
tree
find . -maxdepth 2 -type f
```

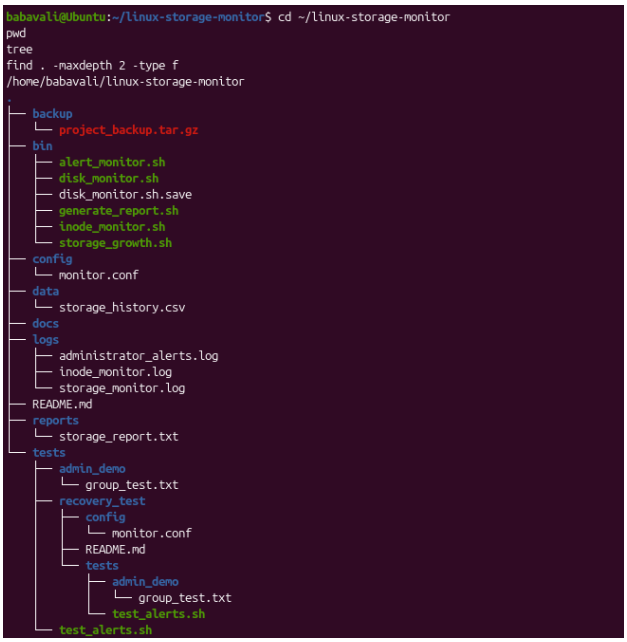


Figure B2 : Ubuntu version and filesystem details

B.3 Storage Monitoring Commands

These commands display filesystem capacity, inode utilization, filesystem type, and project directory size.

```
df -h /
df -i /
df -Th /
du -sh ~/linux-storage-monitor
```

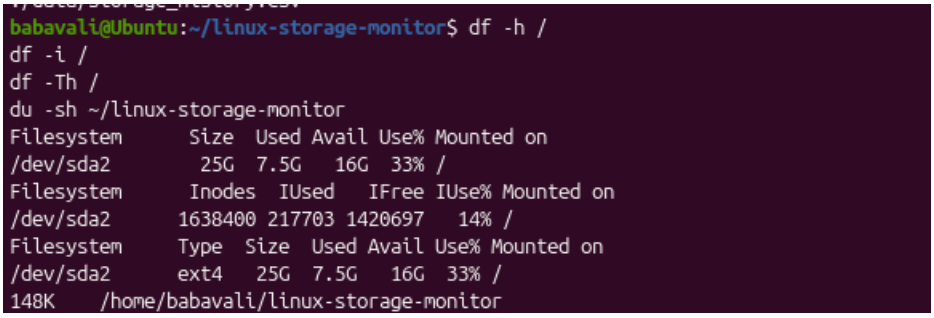


Figure B3 : Project directory structure and installation

B.4 Monitoring Scripts

Bash scripts monitor disk usage, inode usage, storage growth, alerts, and report generation.

```
cd ~/linux-storage-monitor
bash bin/disk_monitor.sh
bash bin/inode_monitor.sh
bash bin/storage_growth.sh
bash bin/alert_monitor.sh
bash bin/generate_report.sh
```

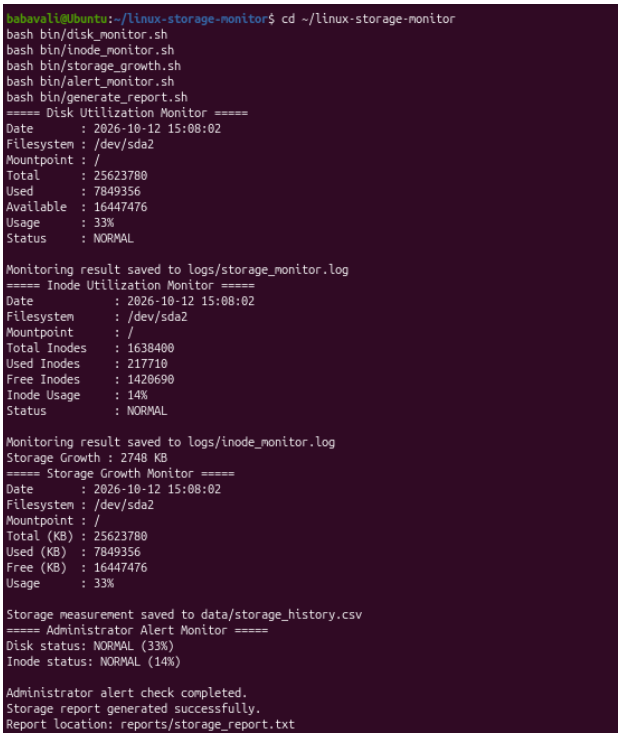


Figure B4 : Project directory structure and installation

B.5 Configuration File

The configuration file defines the monitored filesystem and warning and critical thresholds.

```
cat ~/linux-storage-monitor/config/monitor.conf
```

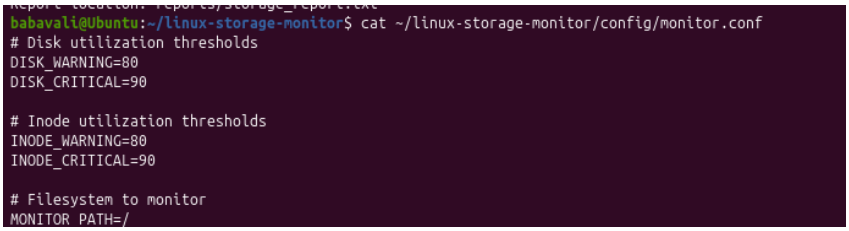


Figure B5 : Project directory structure and installation

B.6 User and Group Management

These commands verify the test user, project group, membership, and shared directory permissions.

```
id storagetest
getent group storageproject
groups storagetest
ls -ld /tmp/storageproject_demo
ls -l /tmp/storageproject_demo
```

.

```
babavali@Ubuntu:~/linux-storage-monitor$ id storagetest
getent group storageproject
groups storagetest
ls -ld /tmp/storageproject_demo
ls -l /tmp/storageproject_demo
uid=1009(storagetest) gid=1009(storagetest) groups=1009(storagetest),100(users),1008(storageproject)
storageproject:x:1008:storagetest
storagetest : storagetest users storageproject
drwxrws--- 2 root storageproject 60 Oct 12 11:54 /tmp/storageproject_demo
```

Figure B6 **:** Project directory structure and installation

B.7 File Permissions and Access Testing

These commands inspect file permissions and test the test user's access to the demonstration file.

```
stat -c '%A %a %n' /tmp/storageproject_demo/group_test.txt
sudo -u storagetest cat /tmp/storageproject_demo/group_test.txt
```

```
babavali@Ubuntu:~/linux-storage-monitor$ stat -c '%A %a %n' /tmp/storageproject_demo/group_test.txt
sudo -u storagetest cat /tmp/storageproject_demo/group_test.txt
stat: cannot stat '/tmp/storageproject_demo/group_test.txt': Permission denied (os error 13)
[sudo: authenticate] Password:
Linux storage permission demonstration
```

Figure B7 **:** Project directory structure and installation

B.8 Package and Process Management

These commands verify the tree package and display running processes and memory usage.

```
tree --version
dpkg -s tree
ps aux | head -n 15
```

```
free -h
top -b -n 1 | head -n 12
```

```
babavali@Ubuntu:~/linux-storage-monitor$ tree --version
dpkg -s tree
ps aux | head -n 15
free -h
top -b -n 1 | head -n 12
tree v2.3.1 © 1996 - 2026 by Steve Baker, Thomas Moore, Francesc Rocher, Florian Sesser, Kyosuke Tokoro
Package: tree
Status: install ok installed
Priority: optional
Section: utils
Installed-Size: 122
Maintainer: Ubuntu Developers <ubuntu-devel-discuss@lists.ubuntu.com>
Architecture: amd64
Version: 2.3.1-1
Depends: libc6 (>= 2.38)
Description: displays an indented directory tree, in color
 Tree is a recursive directory listing command that produces a depth indented
 listing of files, which is colorized ala dircolors if the LS_COLORS environment
 variable is set and output is to tty.
Original-Maintainer: Florian Ernst <florian@debian.org>
Homepage: http://oldmanprogrammer.net/source.php?dir=projects/tree
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.7  0.2  26324 17236 ?        Ss   11:27   1:38 /usr/lib/systemd/systemd --switched-root --system --deserialize=51
root           2  0.0  0.0      0     0 ?        S    11:27   0:01 [kthreadd]
root           3  0.0  0.0      0     0 ?        S    11:27   0:00 [pool_workqueue_release]
root           4  0.0  0.0      0     0 ?        I<   11:27   0:00 [kworker/R-rcu_gp]
root           5  0.0  0.0      0     0 ?        I<   11:27   0:00 [kworker/R-sync_wq]
root           6  0.0  0.0      0     0 ?        I<   11:27   0:00 [kworker/R-kvfree_rcu_reclaim]
root           7  0.0  0.0      0     0 ?        I<   11:27   0:00 [kworker/R-slub_flushwq]
root           8  0.0  0.0      0     0 ?        I<   11:27   0:00 [kworker/R-netns]
root          12  0.0  0.0      0     0 ?        I    11:27   0:00 [kworker/u24:0]
root          13  0.0  0.0      0     0 ?        I<   11:27   0:00 [kworker/R-mm_percpu_wq]
root          14  0.2  0.0      0     0 ?        S    11:27   0:33 [ksoftirqd/0]
root          15  0.1  0.0      0     0 ?        I    11:27   0:22 [rcu_preempt]
root          16  0.0  0.0      0     0 ?        S    11:27   0:00 [rcu_exp_par_gp_kthread_worker/0]
root          17  0.0  0.0      0     0 ?        S    11:27   0:04 [rcu_exp_gp_kthread_worker]
               total        used        free      shared  buff/cache   available
Mem:           6.2Gi       2.3Gi       695Mi        62Mi       3.2Gi       3.8Gi
Swap:             0B          0B          0B
```

Figure B8 : Project directory structure and installation

B.9 Cron Service and Automation

Cron schedules the monitoring scripts and report generation.

```
systemctl is-active cron
systemctl status cron --no-pager
crontab -l
```

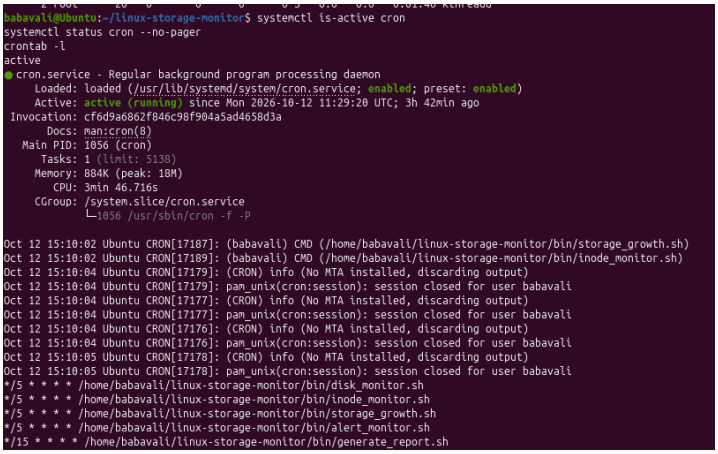


Figure B9 **:** Project directory structure and installation

B.10 Log and Report Management

These commands display recent monitoring logs, alert records, storage history, and the generated report.

```
cd ~/linux-storage-monitor
tail -n 20 logs/storage_monitor.log
tail -n 20 logs/administrator_alerts.log
cat data/storage_history.csv
cat reports/storage_report.txt
```

```
========================================
   LINUX STORAGE MONITORING SYSTEM
========================================

Report generated : 2026-10-12 15:08:02
Filesystem       : /dev/sda2
Mount point      : /

DISK UTILIZATION
----------------
Total space      : 25623780
Used space       : 7849356
Available space  : 16447476
Usage percentage : 33%

INODE UTILIZATION
-----------------
Total inodes     : 1638400
Used inodes      : 217710
Free inodes      : 1420690
Usage percentage : 14%

STORAGE HISTORY
---------------
Measurements recorded : 106

========================================
            END OF REPORT
----------------------------------------
```

Figure B10 : Project directory structure and installation

B.11 Backup and Recovery

The compressed archive is inspected and extracted into a separate directory for recovery testing.

```
cd ~/linux-storage-monitor
ls -lh backup/project_backup.tar.gz
tar -tzf backup/project_backup.tar.gz
```

```
babavali@Ubuntu:~/linux-storage-monitor$ cd ~/linux-storage-monitor
ls -lh backup/project_backup.tar.gz
tar -tzf backup/project_backup.tar.gz
-rw-rw-r-- 1 babavali babavali 1.8K Oct 10 19:53 backup/project_backup.tar.gz
README.md
config/
config/monitor.conf
tests/
tests/admin_demo/
tests/admin_demo/group_test.txt
tests/test_alerts.sh
```

Figure B11 **:** Project directory structure and installation

B.11.1 Recovery Commands

Extract the archive into a separate recovery directory without overwriting the original project.

```
mkdir -p /tmp/storage-monitor-recovery
tar -xzf ~/linux-storage-monitor/backup/project_backup.tar.gz -C /tmp/storage-monitor-recovery
find /tmp/storage-monitor-recovery -type f
```

B.12 Important Project Files

File or Directory	Purpose
bin/disk_monitor.sh	Disk usage monitoring
bin/inode_monitor.sh	Inode usage monitoring
bin/storage_growth.sh	Storage growth tracking
bin/alert_monitor.sh	Alert monitoring
bin/generate_report.sh	Report generation
config/monitor.conf	Monitoring configuration and thresholds
logs/storage_monitor.log	Storage monitoring log
logs/administrator_alerts.log	Administrator alert log
data/storage_history.csv	Storage history records
reports/storage_report.txt	Generated storage report
backup/project_backup.tar.gz	Compressed project backup

TABLE1 : IMPORTANT PROJECT FILES

APPENDIX C: SHELL SCRIPTS

The following Bash scripts demonstrate the main monitoring and reporting functions of the Linux Storage Monitoring System.

C.1 Disk Monitoring Script — bin/disk_monitor.sh

```
#!/bin/bash
# Monitor disk space usage against configured thresholds.

BASE_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
CONFIG_FILE="$BASE_DIR/config/monitor.conf"
LOG_FILE="$BASE_DIR/logs/storage_monitor.log"

if [[ -f "$CONFIG_FILE" ]]; then
    source "$CONFIG_FILE"
else
    echo "Configuration file not found: $CONFIG_FILE" >&2
    exit 1
fi

mkdir -p "$BASE_DIR/logs"
usage=$(df -P "$MONITOR_PATH" | awk 'NR==2 {gsub(/%/,"",$5); print $5}')
timestamp=$(date '+%Y-%m-%d %H:%M:%S')

if [[ -z "$usage" ]]; then
    echo "$timestamp ERROR: Unable to read disk usage for $MONITOR_PATH" | tee -a "$LOG_FILE"
    exit 1
elif (( usage >= DISK_CRITICAL )); then
    echo "$timestamp CRITICAL: Disk usage is ${usage}% on $MONITOR_PATH" | tee -a "$LOG_FILE"
elif (( usage >= DISK_WARNING )); then
    echo "$timestamp WARNING: Disk usage is ${usage}% on $MONITOR_PATH" | tee -a "$LOG_FILE"
else
    echo "$timestamp OK: Disk usage is ${usage}% on $MONITOR_PATH" | tee -a "$LOG_FILE"
fi
```

C.2 Inode Monitoring Script — bin/inode_monitor.sh

```
#!/bin/bash
# Monitor inode usage against configured thresholds.
```

```
BASE_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
CONFIG_FILE="$BASE_DIR/config/monitor.conf"
LOG_FILE="$BASE_DIR/logs/storage_monitor.log"

if [[ -f "$CONFIG_FILE" ]]; then
    source "$CONFIG_FILE"
else
    echo "Configuration file not found: $CONFIG_FILE" >&2
    exit 1
fi

mkdir -p "$BASE_DIR/logs"
usage=$(df -Pi "$MONITOR_PATH" | awk 'NR==2 {gsub(/%/,"",$5); print $5}')
timestamp=$(date '+%Y-%m-%d %H:%M:%S')

if [[ -z "$usage" ]]; then
    echo "$timestamp ERROR: Unable to read inode usage for $MONITOR_PATH" | tee -a "$LOG_FILE"
    exit 1
elif (( usage >= INODE_CRITICAL )); then
    echo "$timestamp CRITICAL: Inode usage is ${usage}% on $MONITOR_PATH" | tee -a "$LOG_FILE"
elif (( usage >= INODE_WARNING )); then
    echo "$timestamp WARNING: Inode usage is ${usage}% on $MONITOR_PATH" | tee -a "$LOG_FILE"
else
    echo "$timestamp OK: Inode usage is ${usage}% on $MONITOR_PATH" | tee -a "$LOG_FILE"
fi
```

C.3 Storage Growth Tracking — bin/storage_growth.sh

```
#!/bin/bash
# Append current storage usage to a CSV history file.

BASE_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
CONFIG_FILE="$BASE_DIR/config/monitor.conf"
HISTORY_FILE="$BASE_DIR/data/storage_history.csv"

if [[ -f "$CONFIG_FILE" ]]; then
    source "$CONFIG_FILE"
else
    echo "Configuration file not found: $CONFIG_FILE" >&2
    exit 1
fi
```

```
mkdir -p "$BASE_DIR/data"
if [[ ! -f "$HISTORY_FILE" ]]; then
   echo "timestamp,mount_point,disk_usage_percent,inode_usage_percent" > "$HISTORY_FILE"
fi

disk_usage=$(df -P "$MONITOR_PATH" | awk 'NR==2 {gsub(/%/,"",$5); print $5}')
inode_usage=$(df -Pi "$MONITOR_PATH" | awk 'NR==2 {gsub(/%/,"",$5); print $5}')
timestamp=$(date '+%Y-%m-%d %H:%M:%S')

if [[ -z "$disk_usage" || -z "$inode_usage" ]]; then
   echo "Unable to collect storage usage for $MONITOR_PATH" >&2
   exit 1
fi

echo "\"$timestamp\",\"$MONITOR_PATH\",$disk_usage,$inode_usage" >> "$HISTORY_FILE"
echo "Storage history updated: $HISTORY_FILE"
```

C.4 Administrator Alert Script — bin/alert_monitor.sh

```
#!/bin/bash
# Record warning and critical storage conditions for administrator review.

BASE_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
CONFIG_FILE="$BASE_DIR/config/monitor.conf"
ALERT_FILE="$BASE_DIR/logs/administrator_alerts.log"

if [[ -f "$CONFIG_FILE" ]]; then
   source "$CONFIG_FILE"
else
   echo "Configuration file not found: $CONFIG_FILE" >&2
   exit 1
fi

mkdir -p "$BASE_DIR/logs"
timestamp=$(date '+%Y-%m-%d %H:%M:%S')
disk_usage=$(df -P "$MONITOR_PATH" | awk 'NR==2 {gsub(/%/,"",$5); print $5}')
inode_usage=$(df -Pi "$MONITOR_PATH" | awk 'NR==2 {gsub(/%/,"",$5); print $5}')

if [[ -z "$disk_usage" || -z "$inode_usage" ]]; then
   echo "$timestamp ERROR: Could not collect storage metrics." | tee -a "$ALERT_FILE"
```

```
    exit 1
fi

if (( disk_usage >= DISK_CRITICAL )); then
    echo "$timestamp CRITICAL: Disk usage ${disk_usage}% on $MONITOR_PATH" | tee -a "$ALERT_FILE"
elif (( disk_usage >= DISK_WARNING )); then
    echo "$timestamp WARNING: Disk usage ${disk_usage}% on $MONITOR_PATH" | tee -a "$ALERT_FILE"
fi

if (( inode_usage >= INODE_CRITICAL )); then
    echo "$timestamp CRITICAL: Inode usage ${inode_usage}% on $MONITOR_PATH" | tee -a "$ALERT_FILE"
elif (( inode_usage >= INODE_WARNING )); then
    echo "$timestamp WARNING: Inode usage ${inode_usage}% on $MONITOR_PATH" | tee -a "$ALERT_FILE"
fi
```

C.5 Report Generation — bin/generate_report.sh

```
#!/bin/bash
# Generate a plain-text storage monitoring report.

BASE_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
CONFIG_FILE="$BASE_DIR/config/monitor.conf"
REPORT_FILE="$BASE_DIR/reports/storage_report.txt"
LOG_FILE="$BASE_DIR/logs/storage_monitor.log"
ALERT_FILE="$BASE_DIR/logs/administrator_alerts.log"
HISTORY_FILE="$BASE_DIR/data/storage_history.csv"

if [[ -f "$CONFIG_FILE" ]]; then
    source "$CONFIG_FILE"
else
    echo "Configuration file not found: $CONFIG_FILE" >&2
    exit 1
fi

mkdir -p "$BASE_DIR/reports"
{
    echo "LINUX STORAGE MONITORING REPORT"
    echo "Generated: $(date '+%Y-%m-%d %H:%M:%S')"
    echo "Monitored path: $MONITOR_PATH"
    echo
    echo "DISK SPACE"
```

```
    df -h "$MONITOR_PATH"
    echo
    echo "INODE USAGE"
    df -i "$MONITOR_PATH"
    echo
    echo "CONFIGURED THRESHOLDS"
    echo "Disk warning: ${DISK_WARNING}%"
    echo "Disk critical: ${DISK_CRITICAL}%"
    echo "Inode warning: ${INODE_WARNING}%"
    echo "Inode critical: ${INODE_CRITICAL}%"
    echo
    echo "RECENT MONITORING LOG ENTRIES"
    if [[ -f "$LOG_FILE" ]]; then tail -n 10 "$LOG_FILE"; else echo "No monitoring log available."; fi
    echo
    echo "RECENT ADMINISTRATOR ALERTS"
    if [[ -f "$ALERT_FILE" ]]; then tail -n 10 "$ALERT_FILE"; else echo "No alert log available."; fi
    echo
    echo "STORAGE HISTORY"
    if [[ -f "$HISTORY_FILE" ]]; then tail -n 10 "$HISTORY_FILE"; else echo "No storage history available."; fi
} > "$REPORT_FILE"

echo "Report generated: $REPORT_FILE"
```

C.6 Monitoring Configuration — config/monitor.conf

```
MONITOR_PATH=/
DISK_WARNING=80
DISK_CRITICAL=90
INODE_WARNING=80
INODE_CRITICAL=90
```

1. Project Overview

The Linux Storage Monitoring System is a small web application built on Ubuntu Linux. It presents disk-space and inode information in a browser dashboard. The backend uses Python Flask to read Linux command output and existing project log/report files, then returns the information to the frontend through HTTP API endpoints.

Main technologies

- Frontend: HTML for page structure, CSS for appearance, and JavaScript for requesting and displaying data.
- Backend: Python with the Flask web framework.
- Linux data sources: df -h / for disk usage and df -i / for inode usage.
- Existing project files: logs/storage_monitor.log and reports/storage_report.txt.
- Local development address: http://127.0.0.1:5000.

2. Frontend and Backend: What Each Part Does

Frontend

The frontend is the web page opened in Firefox. It is stored at backend/templates/index.html. HTML defines the dashboard sections and buttons, CSS styles the cards and layout, and JavaScript calls the backend APIs and puts the returned values on the page.

Backend

The backend is stored at backend/app.py. Flask serves the HTML page and defines API routes. Python runs Linux commands to obtain current disk and inode values and reads the project's log and report files.

How the parts communicate

```
Firefox dashboard (HTML/CSS/JavaScript)
        | fetch('/api/storage') and fetch('/api/logs')
        v
Python Flask backend (backend/app.py)
        | runs df / df -i and reads project files
```

```
        v
Ubuntu Linux + logs/storage_monitor.log + reports/storage_report.txt
```

3. Project Setup

The project was opened in the terminal and the Python virtual environment was activated.

```
cd ~/linux-storage-monitor
source venv/bin/activate
```

The prompt displayed (venv), confirming that the virtual environment was active. Flask was installed in that environment and its version was checked with:

```
python -m flask --version
```

The backend folder and HTML templates folder were created. The two main implementation files are:

```
backend/app.py
backend/templates/index.html
```

A Python syntax check was also performed with python -m py_compile backend/app.py. When this command completed without output, no syntax error was reported.

4. Backend Implementation

The backend file is backend/app.py. Its main responsibilities are:

- Create a Flask application.
- Run df -h / to get disk size, used space, available space, and usage percentage.
- Run df -i / to get total, used, and free inodes and inode usage percentage.
- Serve the dashboard at the root route (/).
- Return storage data as JSON at /api/storage.
- Return recent monitoring log lines as JSON at /api/logs.
- Return the saved storage report as JSON at /api/report.

Backend API routes

Route	Purpose	Result

/	Loads the web dashboard	HTML page
/api/storage	Reads current disk and inode data	JSON
/api/logs	Reads recent monitoring log lines	JSON
/api/report	Reads reports/storage_report.txt	JSON

5. Frontend Implementation

The frontend file is backend/templates/index.html. It includes:

- A page heading identifying the Linux Storage Monitoring System.
- Cards for disk usage, inode usage, and storage status.
- A Storage Information section with a Refresh Storage button.
- A Monitoring Logs section with a Refresh Logs button.
- JavaScript functions loadStorage() and loadLogs() to call the backend APIs.
- Automatic refresh of storage values every 30 seconds while the page is open.

6. Dashboard Result

The dashboard was opened in Firefox at http://127.0.0.1:5000. The screenshot below shows the page rendering live storage values. At the time of the screenshot, disk usage was 34%, inode usage was 14%, and the storage status was NORMAL. These values can change as the system changes.

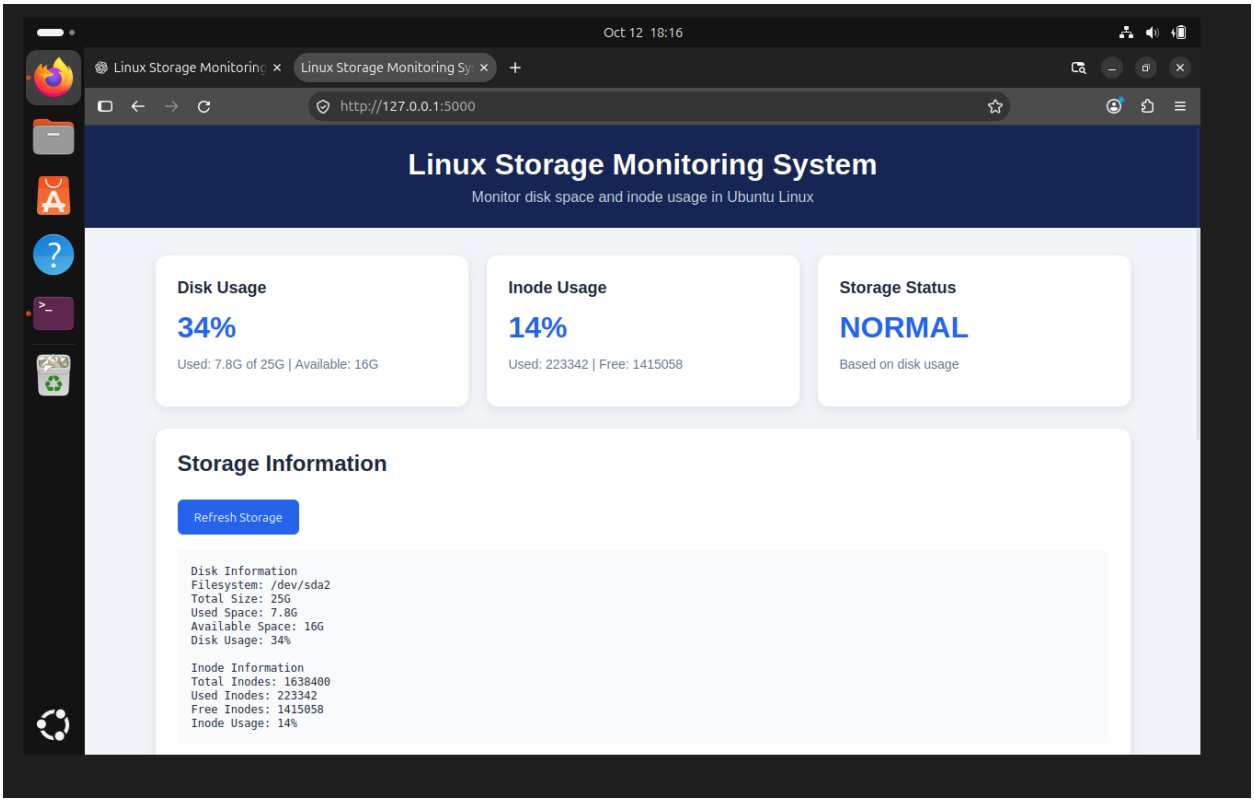


Figure 1. Dashboard displaying disk usage, inode usage, and storage information.

7. Monitoring Logs Display

The Monitoring Logs section calls the /api/logs endpoint and displays recent entries from logs/storage_monitor.log. The screenshot shows timestamps, the monitored filesystem, disk usage percentages, and status values.

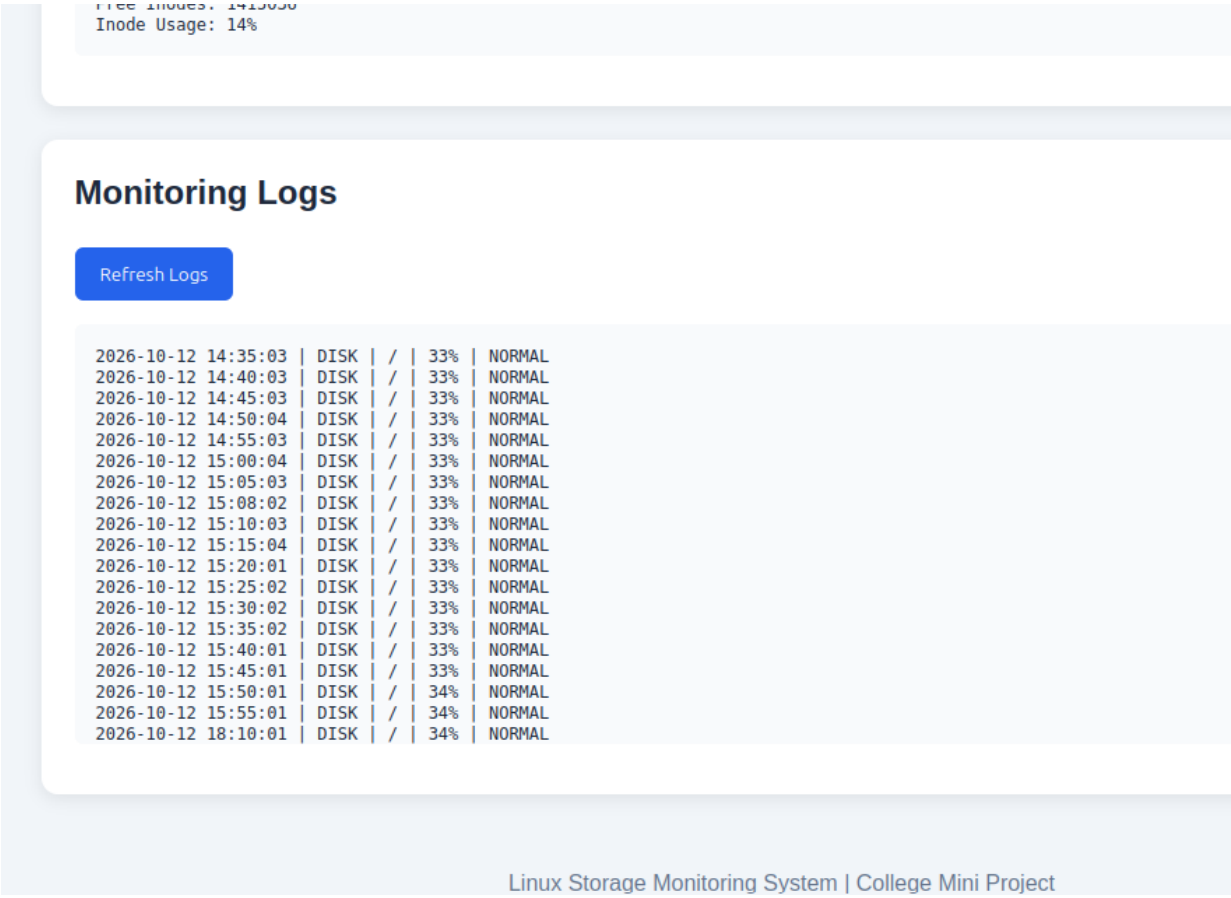


Figure 2. Monitoring log entries displayed in the browser.

8. Testing the Backend APIs

The API endpoints were tested directly from a second terminal using curl. This verifies that Flask returns data independently of the browser display.

Test A: Storage API

```
curl http://127.0.0.1:5000/api/storage
```

The response contained JSON fields for disk filesystem, size, used space, available space, usage percentage, and inode totals and usage.

```
babavali@Ubuntu:~$ curl http://127.0.0.1:5000/api/storage
{
  "disk": {
    "available": "16G",
    "filesystem": "/dev/sda2",
    "size": "25G",
    "usage_percent": "34%",
    "used": "7.8G"
  },
  "inodes": {
    "free_inodes": "1414961",
    "total_inodes": "1638400",
    "usage_percent": "14%",
    "used_inodes": "223439"
  }
}
```

Figure 3. Successful response from the /api/storage endpoint.

Test B: Logs API

```
curl http://127.0.0.1:5000/api/logs
```

The response contained a list of recent monitoring log lines in JSON format.

```
}
babavali@Ubuntu:~$ curl http://127.0.0.1:5000/api/logs
{
  "logs": [
    "2026-10-12 14:40:03 | DISK | / | 33% | NORMAL\n",
    "2026-10-12 14:45:03 | DISK | / | 33% | NORMAL\n",
    "2026-10-12 14:50:04 | DISK | / | 33% | NORMAL\n",
    "2026-10-12 14:55:03 | DISK | / | 33% | NORMAL\n",
    "2026-10-12 15:00:04 | DISK | / | 33% | NORMAL\n",
    "2026-10-12 15:05:03 | DISK | / | 33% | NORMAL\n",
    "2026-10-12 15:08:02 | DISK | / | 33% | NORMAL\n",
    "2026-10-12 15:10:03 | DISK | / | 33% | NORMAL\n",
    "2026-10-12 15:15:04 | DISK | / | 33% | NORMAL\n",
    "2026-10-12 15:20:01 | DISK | / | 33% | NORMAL\n",
    "2026-10-12 15:25:02 | DISK | / | 33% | NORMAL\n",
    "2026-10-12 15:30:02 | DISK | / | 33% | NORMAL\n",
    "2026-10-12 15:35:02 | DISK | / | 33% | NORMAL\n",
    "2026-10-12 15:40:01 | DISK | / | 33% | NORMAL\n",
    "2026-10-12 15:45:01 | DISK | / | 33% | NORMAL\n",
    "2026-10-12 15:50:01 | DISK | / | 34% | NORMAL\n",
    "2026-10-12 15:55:01 | DISK | / | 34% | NORMAL\n",
    "2026-10-12 18:10:01 | DISK | / | 34% | NORMAL\n",
    "2026-10-12 18:15:01 | DISK | / | 34% | NORMAL\n",
    "2026-10-12 18:20:01 | DISK | / | 34% | NORMAL\n"
  ]
```

Figure 4. Successful response from the /api/logs endpoint.

Test C: Report API

```
curl http://127.0.0.1:5000/api/report
```

The response contained the contents of reports/storage_report.txt in a JSON field named report. The report includes filesystem and mount point details, disk utilization, inode utilization, and the number of recorded storage-history measurements.

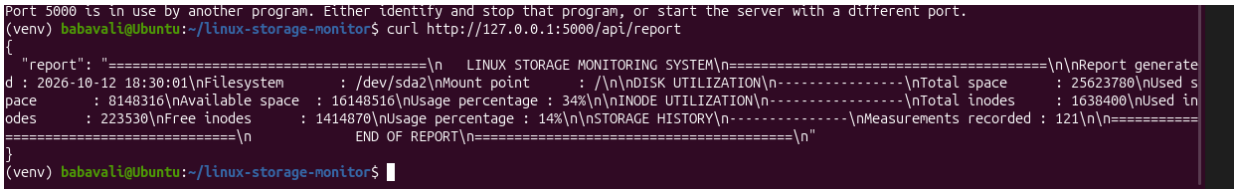


Figure 5. Successful response from the /api/report endpoint.

9. Running the Application

To run the application in Ubuntu:

```
cd ~/linux-storage-monitor
source venv/bin/activate
python backend/app.py
```

Keep the terminal running the Flask server open. Then open Firefox and visit **http://127.0.0.1:5000**. To test the APIs, open a second terminal and run one of the curl commands in Section 8.

If Flask displays 'Address already in use' for port 5000, another process may already be running the server. If the dashboard and API requests are already working, do not start a second copy unnecessarily.

10. Current Results and Limitations

- The frontend dashboard loaded successfully in Firefox.
- Disk and inode data were displayed from the Ubuntu system.
- The monitoring log section displayed existing log records.
- The /api/storage and /api/logs endpoints were tested successfully with curl.
- The /api/report endpoint was added and tested successfully with curl.
- The report API returns the report text as JSON; a report button on the frontend has not yet been connected.

- The project was run as a local development application. Flask's built-in development server is suitable for this demonstration, not production deployment.

11. Simple Explanation for Project Demonstration

"Our project is a Linux Storage Monitoring System. The frontend is built using HTML, CSS, and JavaScript. It shows disk usage, inode usage, storage status, and monitoring logs. The backend is developed in Python using Flask. It runs Linux commands such as df -h and df -i to obtain storage information. The frontend requests this information from Flask API endpoints, and the backend returns the results in JSON format. We tested the storage and logs APIs using curl, and also added and tested an API to retrieve our existing storage report."

12. Important File Locations

```
linux-storage-monitor/
├── backend/
│   ├── app.py
│   └── templates/
│       └── index.html
├── logs/
│   └── storage_monitor.log
└── reports/
    └── storage_report.txt
```